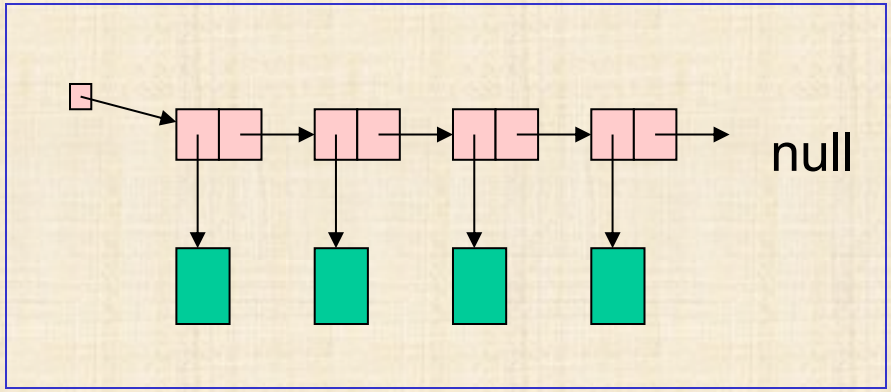
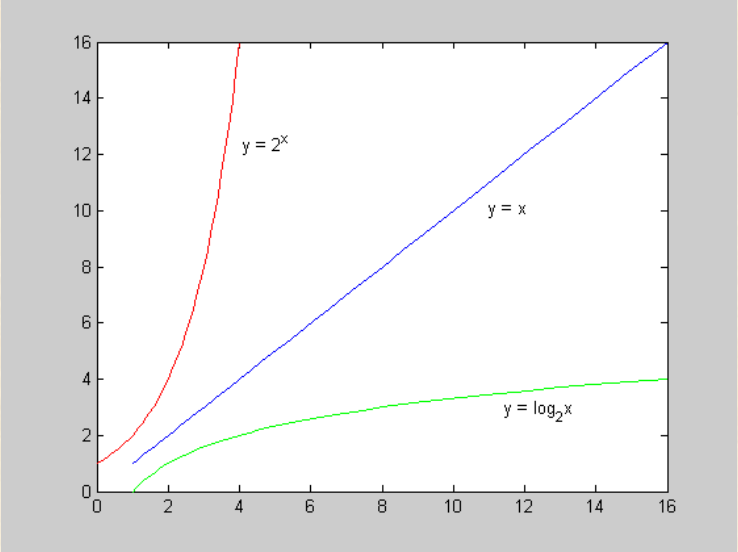
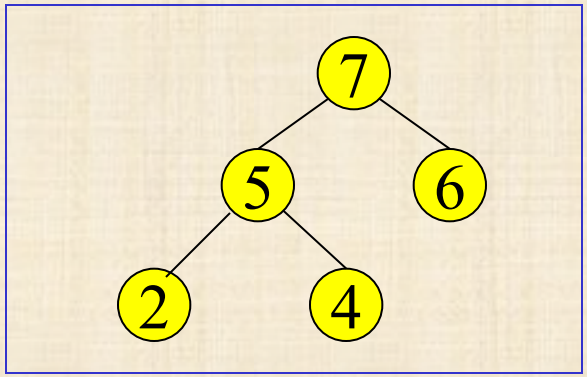
Welcome to Data Structures
Spring 2026







Welcome to Data Structures Spring 2026

במקרה אזעקה:

- אם האזעקה אצל המרצה – השיעור יופסק עד חזרה לשגרה
- אם האזעקה רק אצל הסטודנטים:
  - סטודנט יסמן "הרמת יד" וילך להתמגן
  - במידה ויותר מ-30% מהסטודנטים מתמגנים – השיעור יופסק עד לחזרה לשגרה.

- בכל מקרה, השאירו את ה-zoom פתוח עד החזרה – כדי לחסוך זמן התחברות חוזר.

- גם במידה ומשך ההתגוננות עובר את מועד סוף השיעור – ניפגש ל-2 דקות לסגור את השיעור בצורה מסודרת ולהודעות.

Welcome to Data Structures Spring 2026

In Case of Siren

- If the siren sounds at the instructor's location – the class will be paused until the situation returns to normal.
- If the siren sounds only for the students:
  - A student should use the "Raise Hand" feature and go to a protected area.
  - If more than 30% of the students take shelter – the class will be paused until the situation returns to normal.

- Please keep the Zoom session open until we return, to avoid reconnection delays.
- Even if the sheltering period extends beyond the scheduled end of the class, we will reconnect for two minutes to close the session properly and share any final announcements.

Welcome to Data Structures Spring 2026

- Lecturer: Shmulik kaplan
  - E-mails: shmuel.kaplan01@runi.ac.il
- TAs: Itamar Dikstein, Roi Mashiah and Jonathan Simon
- Course resources:
  - Moodle: Panopto, HW submission only
  - Piazza: Q&A forum, lecture notes, recitations, HW files.

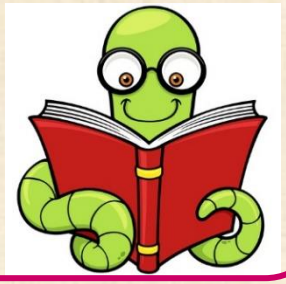
Student's Responsibilities

Submit assignments:

6 theoretical (dry)

3 programming (wet)

Read



Answer questions during class

Ask, if you do not understand

Final Grade

- If final exam < 60 then

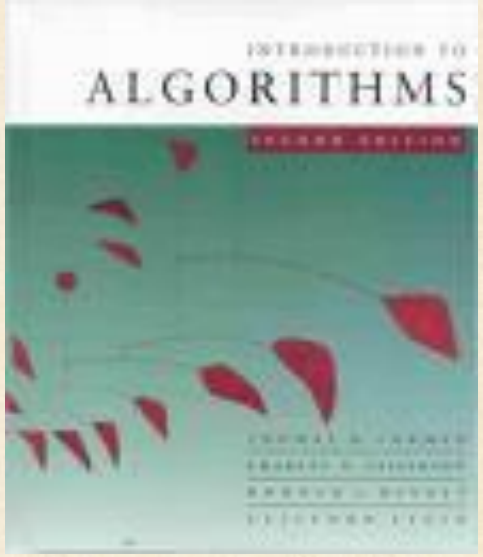
  grade = fail (sorry)

  else

  grade = 0.15 * HW + 0.85 * final exam
- All assignments count towards the grade
- Attendance bonus:
  - All lectures = 3pts
  - All lectures but 1 = 2 pts
  - All lectures but 2 = 1 pt

Textbook

- Cormen, Leiserson, Rivest and Stein *Introduction to Algorithms. (CLRS)*
- *A Hebrew translation exists (by Open univ.)*



Ethics

- Collaboration in groups of 2-3 students is encouraged!
- But – take no written notes, no screenshots, no pictures, and write your own solution in your own words on your own.

Ethics

- Collaboration in groups of 2-3 st ents is encouraged!
- But – take no wri screensh and write your o own words on your ow

if you cannot reconstruct it by yourself, you do not own it.

LLM Policy
(ChatGPT and its kind)

- The material of this course is so basic that it is likely that LLM will produce excellent results.
- However, you will not learn the basics by using LLMs.
- Failing to understand the basics at this level will cause non-bridgeable gaps in later more complex courses!

LLM Policy

(ChatGPT and its kind)

- The material of this course i sic that it is likely that LL excellent res
- How basics b
- Fa rstand the basics at this leve induce none bridgeable gaps in later more complex courses!

It is forbidden to use LLMs to produce any solutions to problems and exercises

Credits

- The slides in this course include slides by O. Weimann, T. Tamir, A. Tal, A. Shamir, Y. Moses.

Course Goals

- You will know and understand the basics of algorithms, data structures and asymptotic analysis.
- Introduce you into the computational lens (way of thinking) of theoretical computer science.
- Enhance your abilities to understand and articulate complex technical concepts using clear, common and precise language.

Today

- What is a data structure?
- What is it good for?

Solving a Task (a problem)

Input

↓

manipulation

↓

Output

Algorithm

Data {input} 2,1,12,7

↓ ↓ ↙

algorithm

↓ ↓ ↓

Data {output} 1,2,7,12 Shimon

Solving a Task (a Problem)

Data: Information for analysis.

e.g., numbers, words, songs, movies

Algorithm: A well-defined procedure to solve a problem

Data structure: The way the data is organized

Problem Examples

- Data: a set of songs
- Possible tasks:
  - Find a song by its name
  - Find a song by its length
  - Find a song by its first notes
  - Find the preceding song according to recording date
  - Find the 10 most popular songs
  - Execute an “echo” filter to make songs fill like they are played in a big arena

Another Example

- Given:
  - Yossi is the father of Dan and Ben
  - Ben is a father of Ran, Gal, and Avi
  - Dan is a father of Gil and Bil
  - Gil is a father of Seffi and Ruthi
- Who is the grandfather of Seffi?

Considerations

- How to keep/organize the data?
  - How fast is the computation of a task?
  - How fast is it to update the data?
  - How much space is used to store the data?
- How do we measure time and/or space?
  - Do we care about a single operation or a sequence of operations?
  - Do we care about the worst case, best case, average case?

The Sorting Problem

Input: A sequence of n numbers $<a_1, a_2, .., an>$

Output: A permutation (reordering) $<b_1, b_2, .., b_n>$ of the input sequence satisfying $b_1 \leq b_2 \leq \cdots \leq b_n$

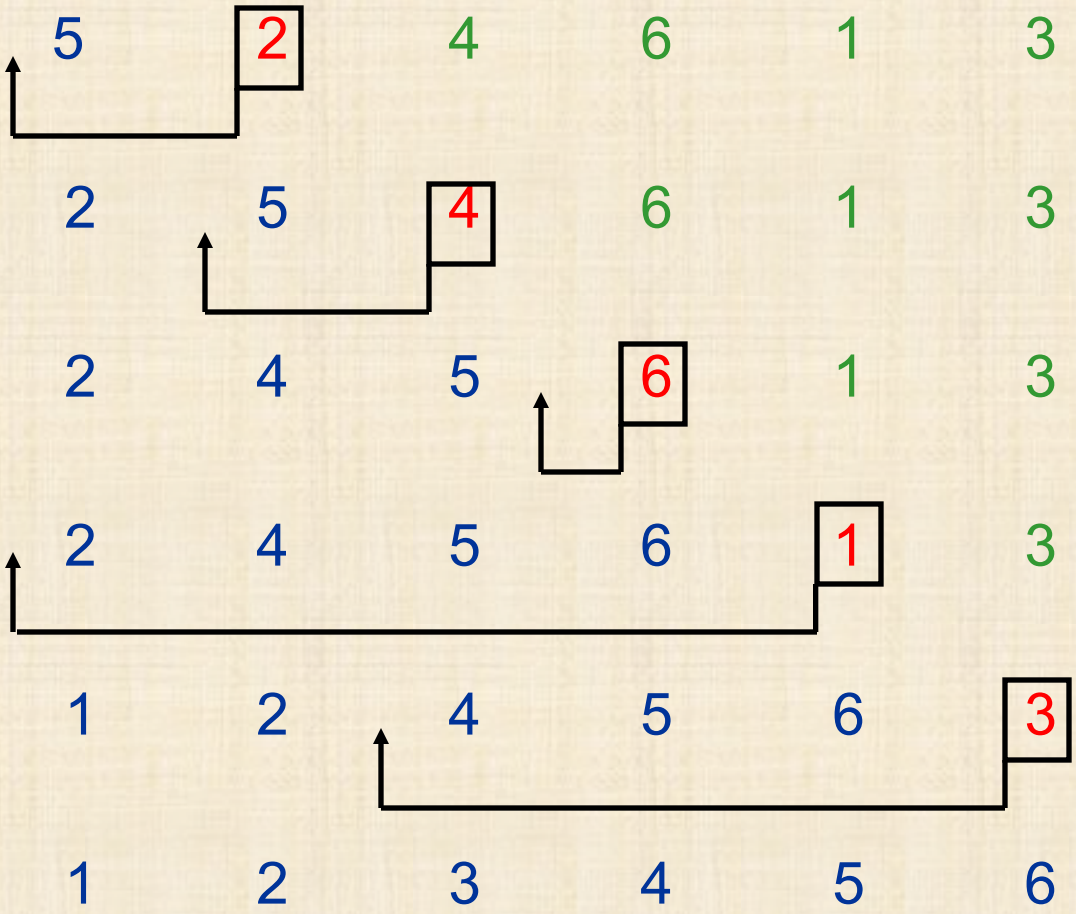
Can we sort only numbers?

Insertion Sort

- Input – Output:
  - Numbers, images, songs, movies, names
  - Definition / specification of order
- How do we describe the algorithm?

Insertion Sort: by example

visualization



Insertion Sort Description 1st Attempt

- Go over the elements according to their position in the sequence
- For each element, insert it into the correct place in the already sorted prefix of elements preceding it
- Until last element inserted into place

What is missing?

- How are the data (numbers) represented:
  - Each element: number - binary
  - The sequence: ?
- What are the operations required from the chosen representation?
  - Go over the elements according to their order in the sequence
  - For each element, insert it into correct place in the already sorted prefix of elements preceding it
  - Until last element inserted into place

Insertion Sort Description 2nd Attempt

(with an array) Java code

```
public static void insertionSort (int[] array){
    for (int j = 1; j < array.length; j++) {
        int key = array[j];
        int k = j - 1;
            while(k >= 0 && array[k] > key) {
                array[k+1] = array[k]; k--;
            }
            array[k+1] = key;
    }
}
```

- Go over the elements according to their order in the sequence
- For each element, insert it into correct place in the already sorted prefix of elements preceding it
- Until last element inserted into place

Insertion Sort Description
2nd Attempt
(with list) scheme code

```
(define (insertion-sort lst)
  (if (null? lst)
      '()
      (insert (car lst) (insertion-sort (cdr lst)))))

(define (insert val sorted-lst)
  (cond ((null? sorted-lst) (list val))
        ((<= val (car sorted-lst)) (cons val sorted-lst))
        (else (cons (car sorted-lst) (insert val (cdr sorted-lst))))))
```

- Go over the elements according to their order in the sequence
- For each element, insert it into correct place in the already sorted prefix of elements preceding it
- Until last element inserted into place

Insertion Sort Description 2nd Attempt

(with an array) Java code

```
public static void insertionSort (int[] array){
    for (int j = 1; j < array.length; j++) {
        int key = array[j];
        int k = j - 1;
            while(k >= 0 && array[k] > key) {
                array[k+1] = array[k]; k--;
            }
            array[k+1] = key;
    }
}
```

- Go over the elements according to their order in the sequence
- For each element, insert it into correct place in the already sorted prefix of elements preceding it
- Until last element inserted into place

Insertion Sort 3rd Attempt

Pseudocode (with an array)

```
Insertion-Sort(A)
  for  i ← 2  to  length[A]
          key ← A[i]
          j ← i-1
          while  j>0  and  A[j]>key
                    A[j+1] ← A[j]
                    j ← j-1
          end
            A[j+1] ← key
  end
```

5	2	4	6	1	3

What is the best case?

What is the worse case?

Note: In this example, the first index of A is 1

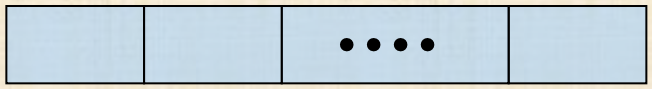
Pseudocode

- Precise description of an algorithm
- Independent of a specific programing language
- Usually easy to translate to real code
- Used in CS literature
- Will be used in class and HW
- Read by a person – not by a computer

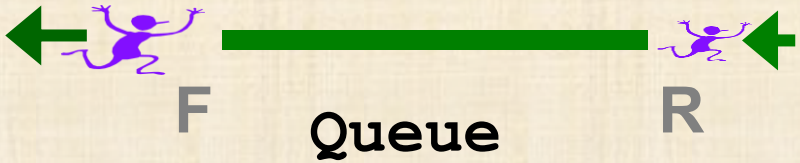
What is a Data Structure?

The way the data is organized

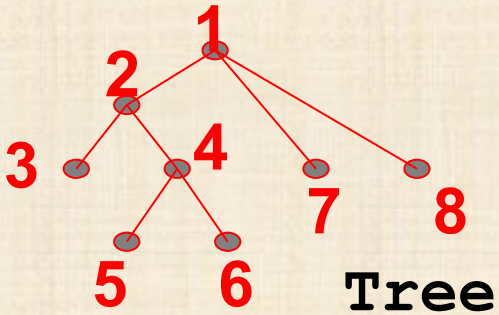
Elementary Data Structures



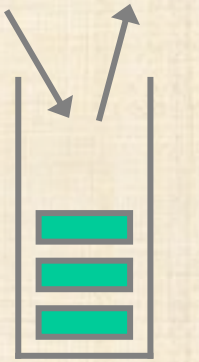
Array



Queue



Tree



Stack

Linked list

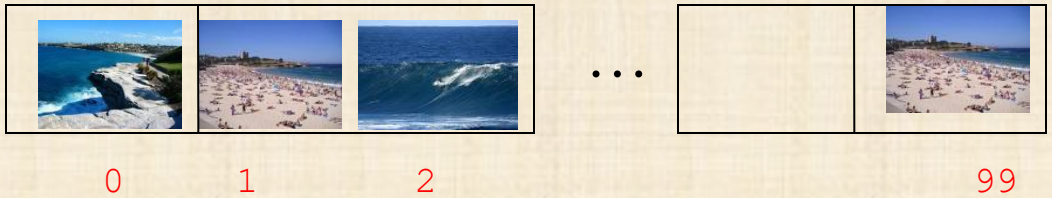
some languages include standard implementations, others do not

Examples

- Arrays
  - A[0..99] : integer array

	0	1	2	3	4	5	6	7					99
A	2	1	3	3	2	9	9	6	…				10

  - A[0..99] : array of images



Example: A Sequence

$$[a_0\, a_1\, a_2\, a_3\, \ldots\, a_{i\text{-}1}\, \overset{?}{a_i}\, a_{i+1}\, \ldots\, a_{n\text{-}2}\, a_{n\text{-}1}\,]$$

Retrieve the item at position i

$$[a_0\, a_1\, a_2\, a_3\, \ldots\, a_{i\text{-}1}\, \overset{b}{\frown}\, a_i\, a_{i+1}\, \ldots\, a_{n\text{-}2}\, a_{n\text{-}1}\,]$$

Insert b at position i

$$[a_0\, a_1\, a_2\, a_3\, \ldots\, a_{i\text{-}1}\, \cancel{a_i}\, a_{i+1}\, \ldots\, a_{n\text{-}2}\, a_{n\text{-}1}\,]$$

Delete the item at position i

Abstract Data Type (ADT)

- An abstract (mathematical) specification of objects (values) and their functionality (operations)
- From the point of view of the user
- Does not specify the implementation

Abstract Data Type (ADT) example: Sequence

- represents a sequence of items

Insert(S,i,b) – Insert b as the i-th item of S
Retrieve(S,i) – Return the i-th item of S
Delete(S,i) – Delete and return the i-th item of S

S = Sequence() – Return an empty sequence
Length(S) – Return the number of items in S

Interesting special cases:

Insert-First(S,b), Retrieve-First(S), Delete-First(S)
Insert-Last(S,b), Retrieve-Last(S), Delete-Last(S)

Stacks, Queues, Deques

Stack – Supports only:

Sequence(), Insert-Last(*S*,*b*), Retrieve-Last(*S*), Delete-Last(*S*)

Also known as: Stack(), Push(*S*,*b*), Top(*S*), Pop(*S*)

Last In First Out (LIFO)

Queue – Supports only:

Sequence(), Insert-Last(*Q*,*b*), Retrieve-First(*Q*), Delete-First(*Q*)

Also known as: Queue(), Enqueue(Q,b), Dequeue(Q,b)

First In First Out (FIFO)

Deque (double ended queues) – Supports only:

Sequence(), Insert-First(*D*,*b*), Retrieve-First(*D*), Delete-First(*D*)

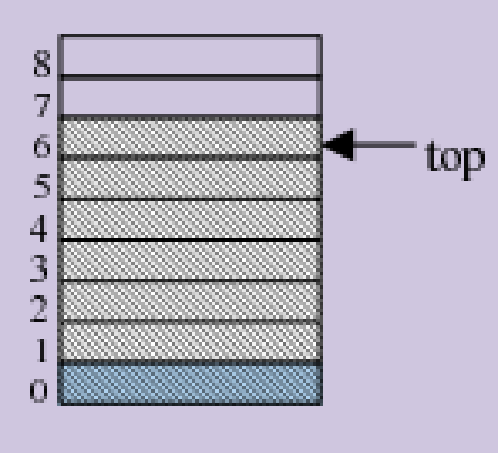
Deque(), Insert-Last(*D*,*b*), Retrieve-Last(*D*), Delete-Last(*D*)

ADT vs. Data Structure vs. Implementation

- ADT = Specification of the abstract objects and their functionality
- Data Structure = A logical structure and set of procedures implementing an ADT
- Implementation of a data structure = Concrete objects and procedures describing the actual implementation of the data structure and performing its functionality

Question

- What is the following description of a stack?
  - An array with a pointer to top
    - Initialize: top = 0;
    - Push x: top=top+1; A[top]=x;
    - Top: A[top];
    - Pop: top = top-1;



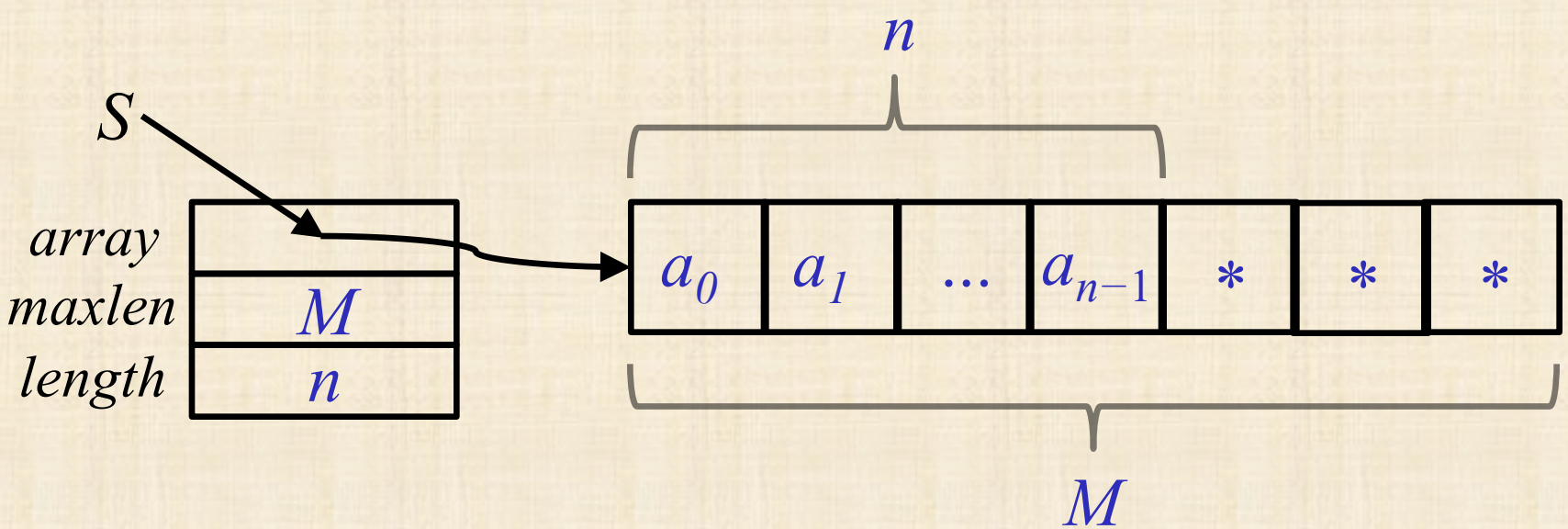
This is an <u>implementation</u> of the stack ADT using an array

ADT vs. Implementation

- User knows the specification, not the implementation
- Implementation can change over time
- The quality of the implementation:
  - Time complexity (query / update)
  - Space complexity
  - preprocessing (setup) time
  - Simplicity of programming / the underlying algorithms

Implementing the Sequence ADT using an array

We need to know the maximal length M in advance.



Retrieve(S,i) (of any element) takes O(1) time

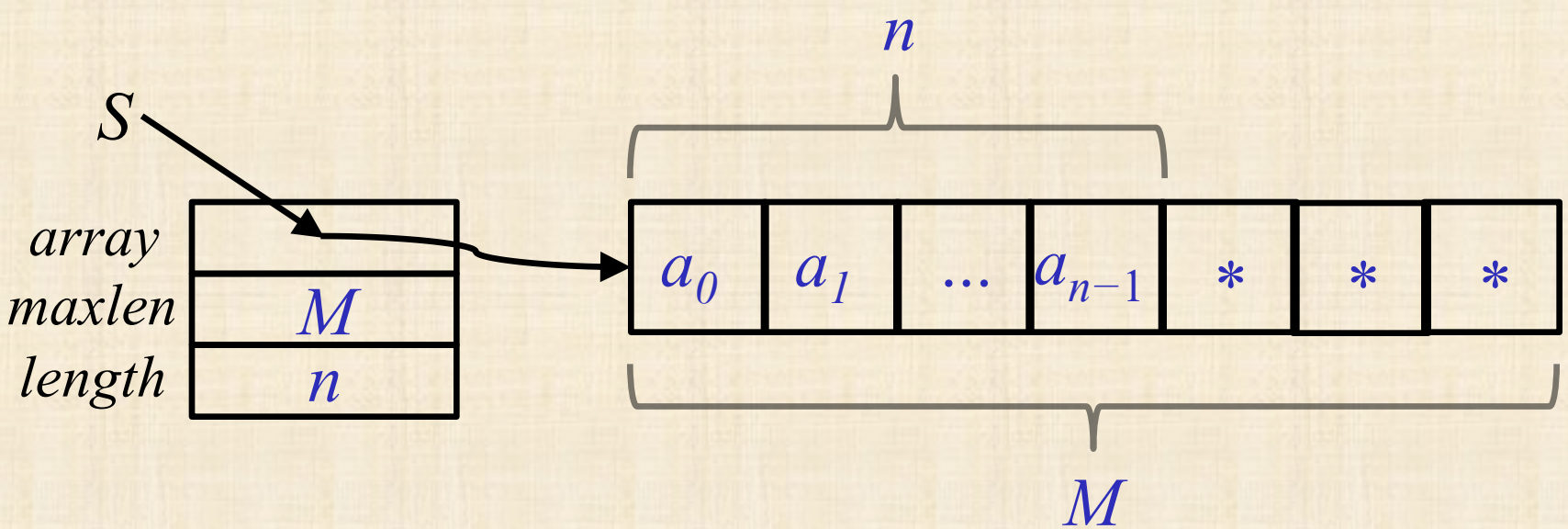
Insert-Last(S,b) and Delete-Last(S) take O(1) time

we will define O-notation formally later on

Stack operations in O(1) time

Implementing Sequence using an array

We need to know the maximal length M in advance.



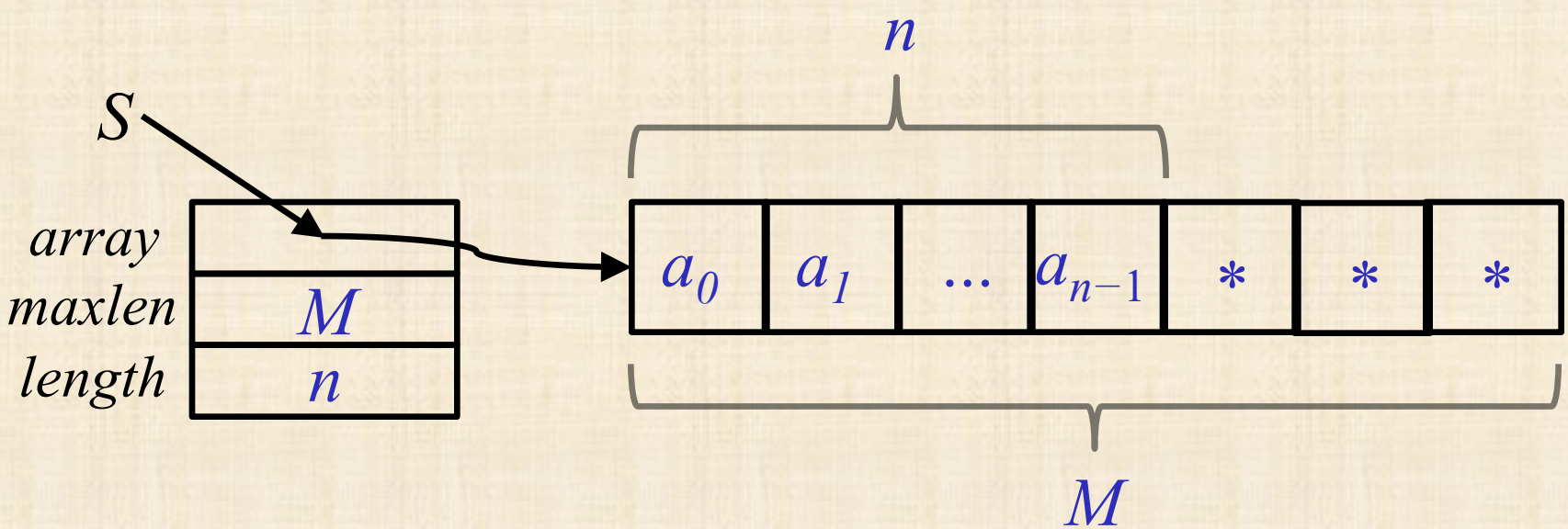
Retrieve(S,i) (of any element) takes O(1) time

Insert-Last(S,b) and Delete-Last(S) takes O(1) time

Insert(S,i,b) and Delete(S,i)?

Implementing Sequence using an array

We need to know the maximal length M in advance.

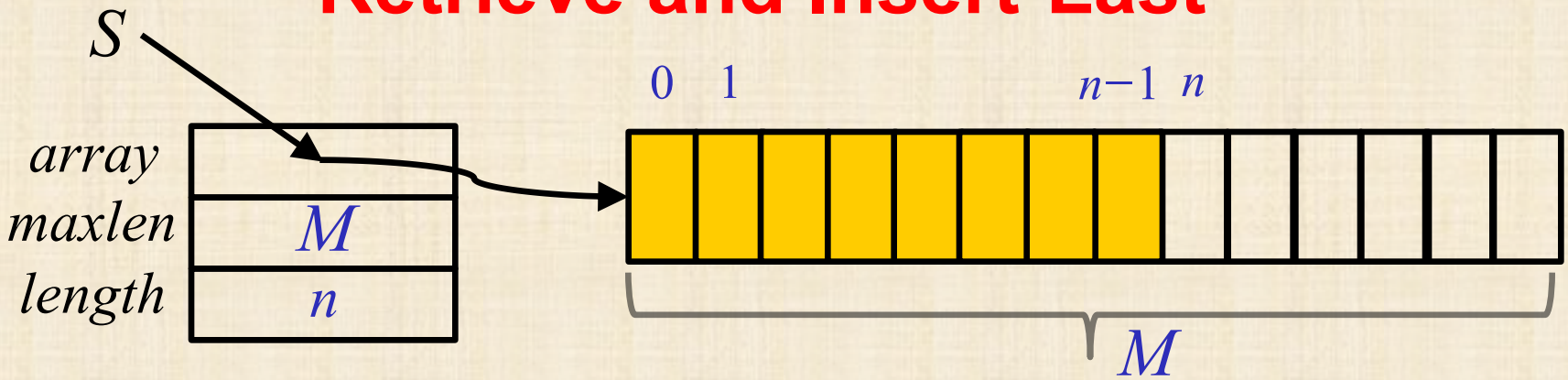


Retrieve(S,i) (of any element) takes O(1) time

Insert-Last(S,b) and Delete-Last(S) takes O(1) time

Insert(S,i,b) and Delete(S,i)? takes O($n-i+1$) time

Implementing Sequence using an array Retrieve and Insert-Last



Function `Retrieve`(S, i)

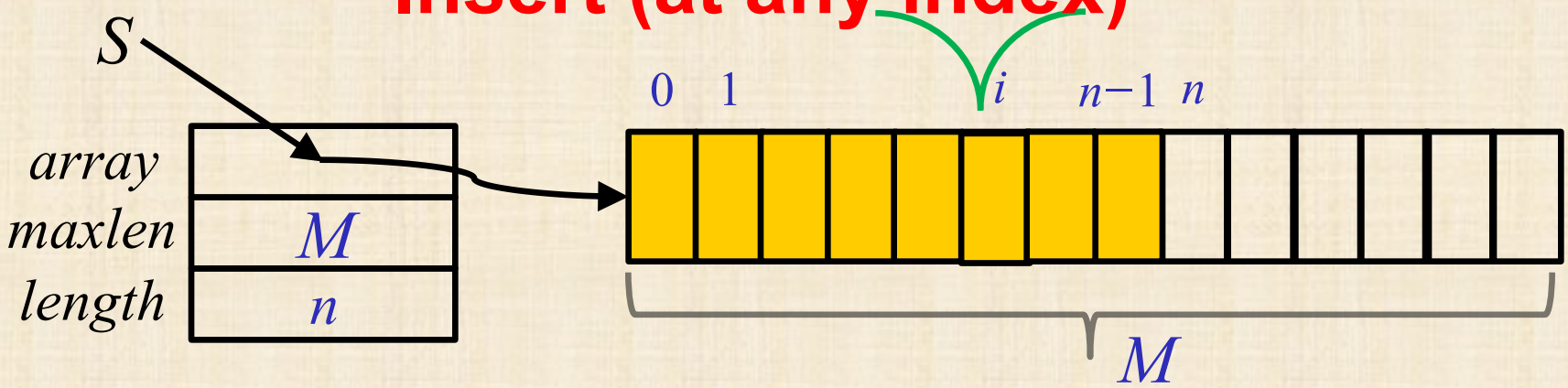
if $i < 0$ **or** $i \geq S.length$ **then error**
return $S.array[i]$

Function `Insert-Last`(S, b)

if $S.length = S.maxlen$ **then error**
$S.array[S.length] \leftarrow b$
$S.length \leftarrow S.length + 1$

Delete-Last very similar

Implementing Sequence using an array
Insert (at any index)



Function `Insert`(S, i, b)

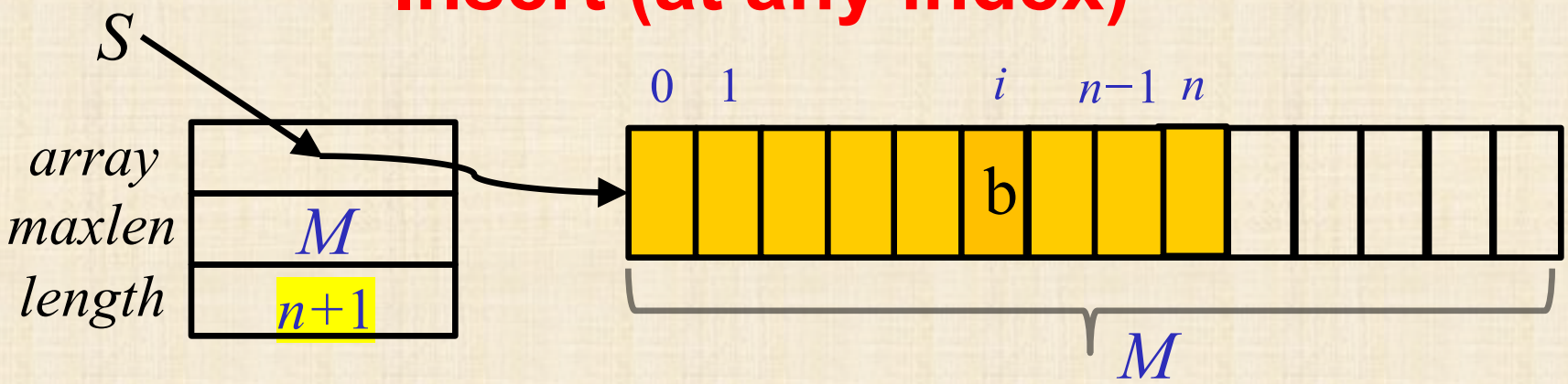
if $S.length = S.maxlen$ **then error**

for $j \leftarrow S.length - 1$ **downto** i **do**
  $S.array[j+1] \leftarrow S.array[j]$

$S.array[i] \leftarrow b$

$S.length \leftarrow S.length + 1$

Implementing Sequence using an array
Insert (at any index)



Function `Insert`(S, i, b)

if $S.length = S.maxlen$ **then error**

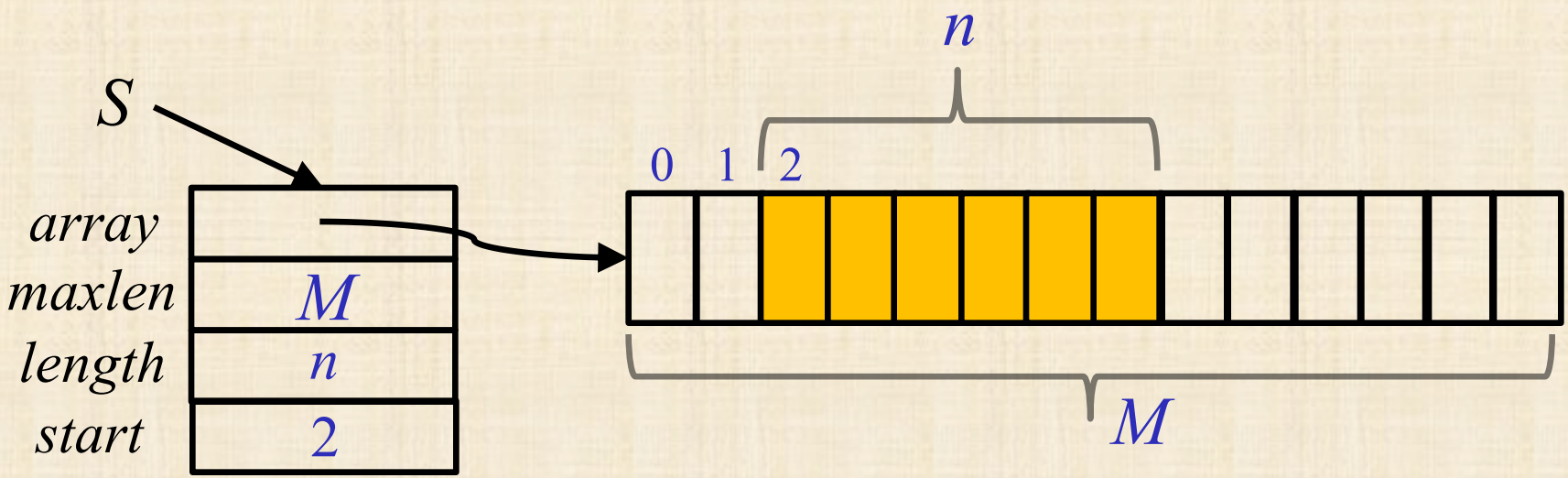
for $j \leftarrow S.length - 1$ **downto** i **do**
  $S.array[j+1] \leftarrow S.array[j]$

$S.array[i] \leftarrow b$

$S.length \leftarrow S.length + 1$

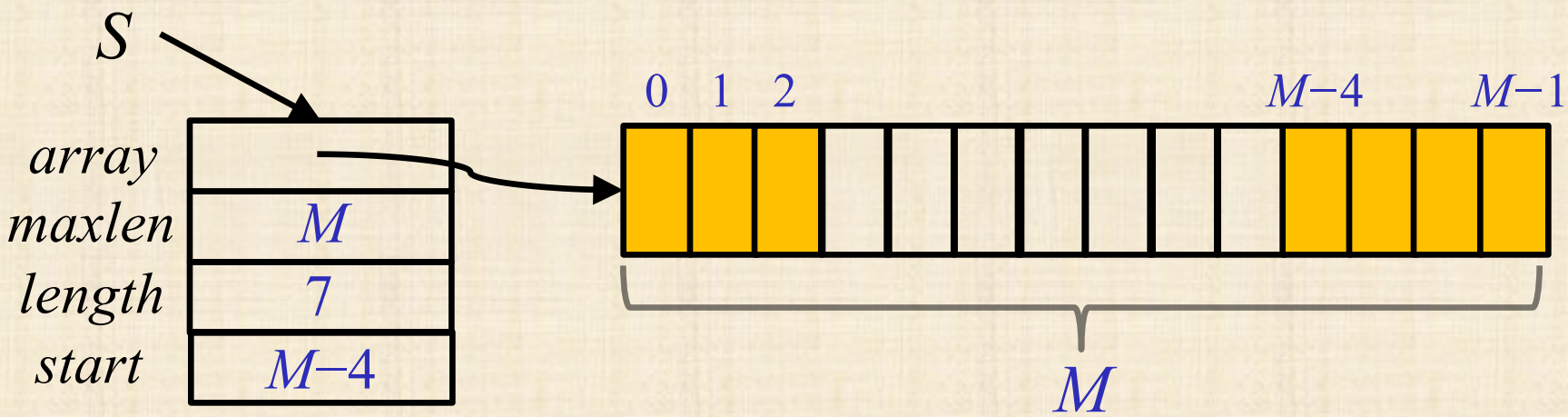
We need to move $n-i$ items, then insert. $O(n-i+1)$ time.

Implementing Sequence using a circular array



New field: *start*

Implementing Sequence using a circular array - Retrieve

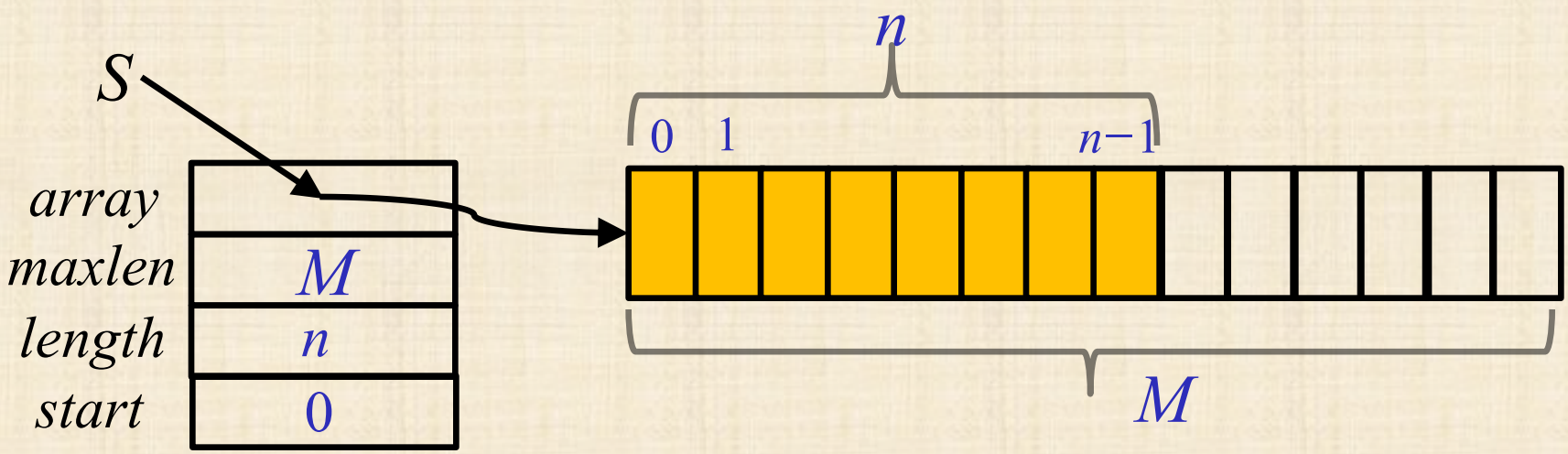


Occupied region can wrap around!

Function `Retrieve`(S, i)

if $i < 0$ **or** $i \geq S.length$ **then error**
return $S.array[(S.start + i) \bmod S.maxlen]$

Implementing Sequence using a circular array – Delete-First



Function `Delete-First`(S)

if $S.length = 0$ **then error**

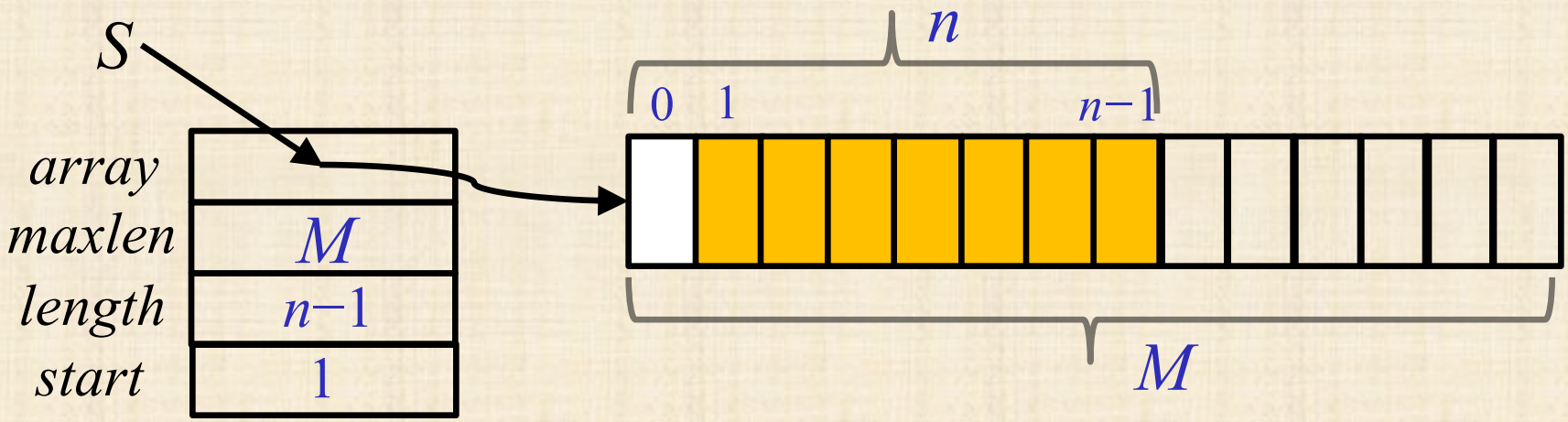
$b \leftarrow S.array[S.start]$

$S.start \leftarrow (S.start + 1) \bmod S.maxlen$

$S.length \leftarrow S.length - 1$

return b

Implementing Sequence using a circular array – Delete-First



Code of other operations similar

Function `Delete-First`(S)

if $S.length = 0$ **then error**

$b \leftarrow S.array[S.start]$
$S.start \leftarrow (S.start + 1) \bmod S.maxlen$

$S.length \leftarrow S.length - 1$

return b

Reminder

Stack – Supports only:

Sequence(), Insert-Last(*S*,*b*), Retrieve-Last(*S*), Delete-Last(*S*)

Also known as: Stack(), Push(*S*,*b*), Top(*S*), Pop(*S*)

Last In First Out (LIFO)

Queue – Supports only:

Sequence(), Insert-Last(*Q*,*b*), Retrieve-First(*Q*), Delete-First(*Q*)

Also known as: Queue(), Enqueue(Q,b), Dequeue(Q,b)

First In First Out (FIFO)

Deque (double ended queues) – Supports only:

Sequence(), Insert-First(*D*,*b*), Retrieve-First(*D*), Delete-First(*D*)

Deque(), Insert-Last(*D*,*b*), Retrieve-Last(*D*), Delete-Last(*D*)

Conclusion

- We have just implemented all functionality needed for:
  - Stack
  - Queue
  - Deque
- In O(1) for each operation

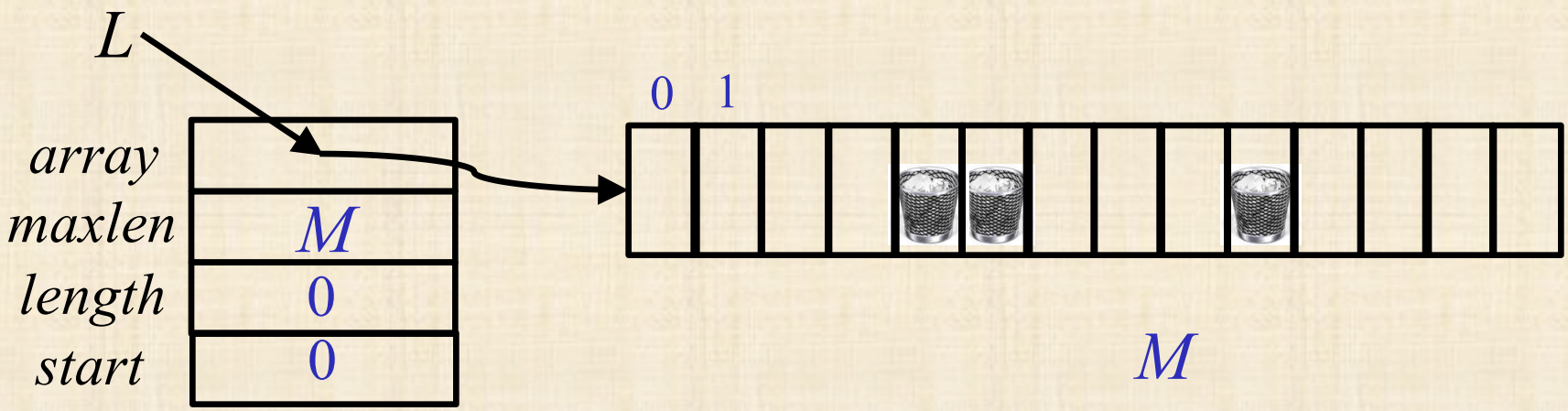
Conclusion

- We have just implemented all functionality needed for:
  - Stack
  - Queue
  - Deque
- In O(1) for each operation

Have we?!

Implementing Sequence using a circular array

What about the initialization of the data structure?



Do we really need O(M) time to initialize *array* ?

OS says where *array* begins,
array may contain garbage

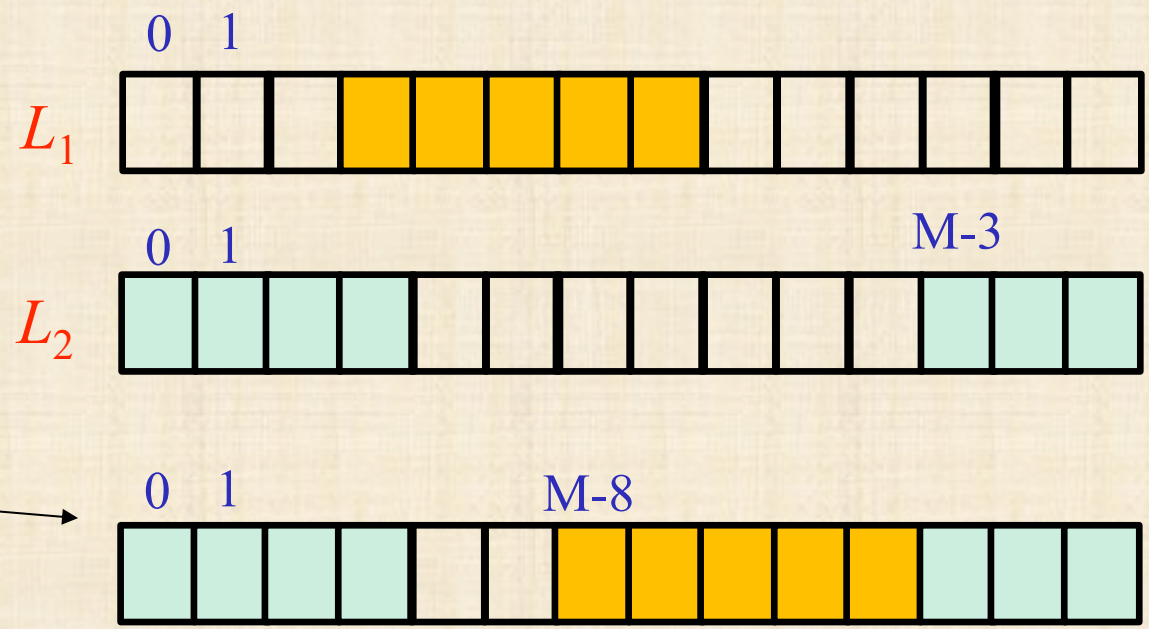
Concatenating Sequences using circular arrays

Concat(L_1,L_2) – Return a list consisting of the elements of L_1 followed by the elements of L_2.

Assume that $|L_1| + |L_2| \leq M$

$n_1 = 5, n_2 = 7$

$n_1 < n_2$ *so copy* L_1 *before* L_2



Concat(L_1,L_2)

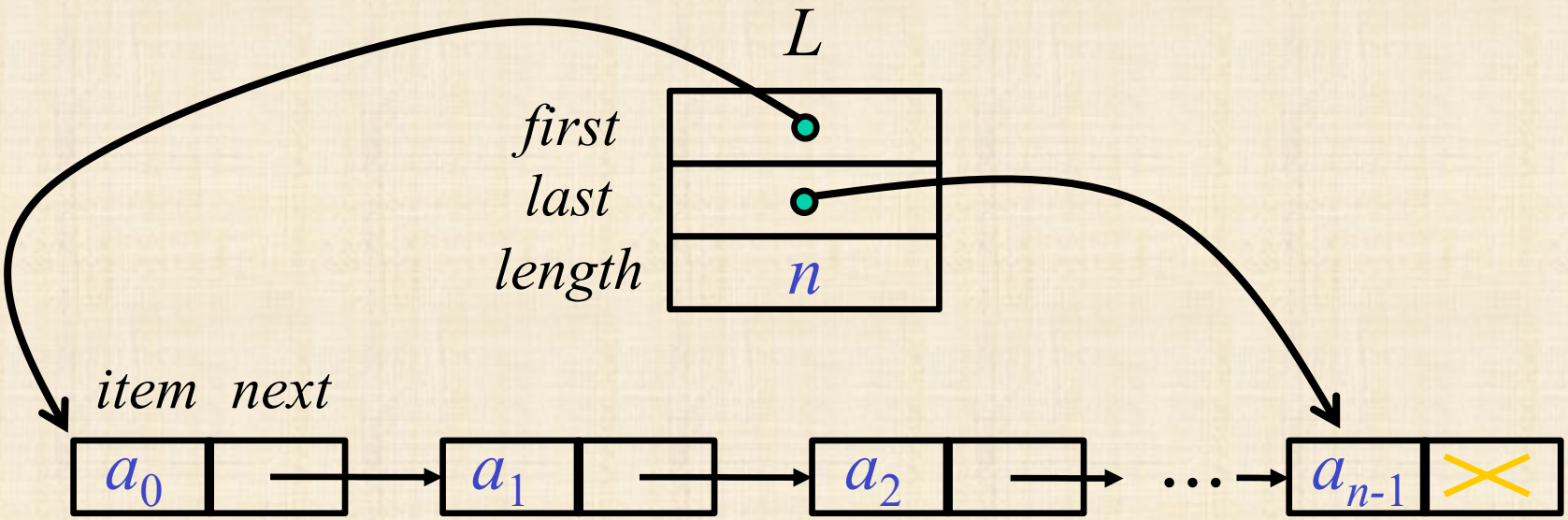
array	
maxlen	M
length	$n_1 + n_2$
start	$M - 8$

$O(\min\{n_1, n_2\} + 1)$

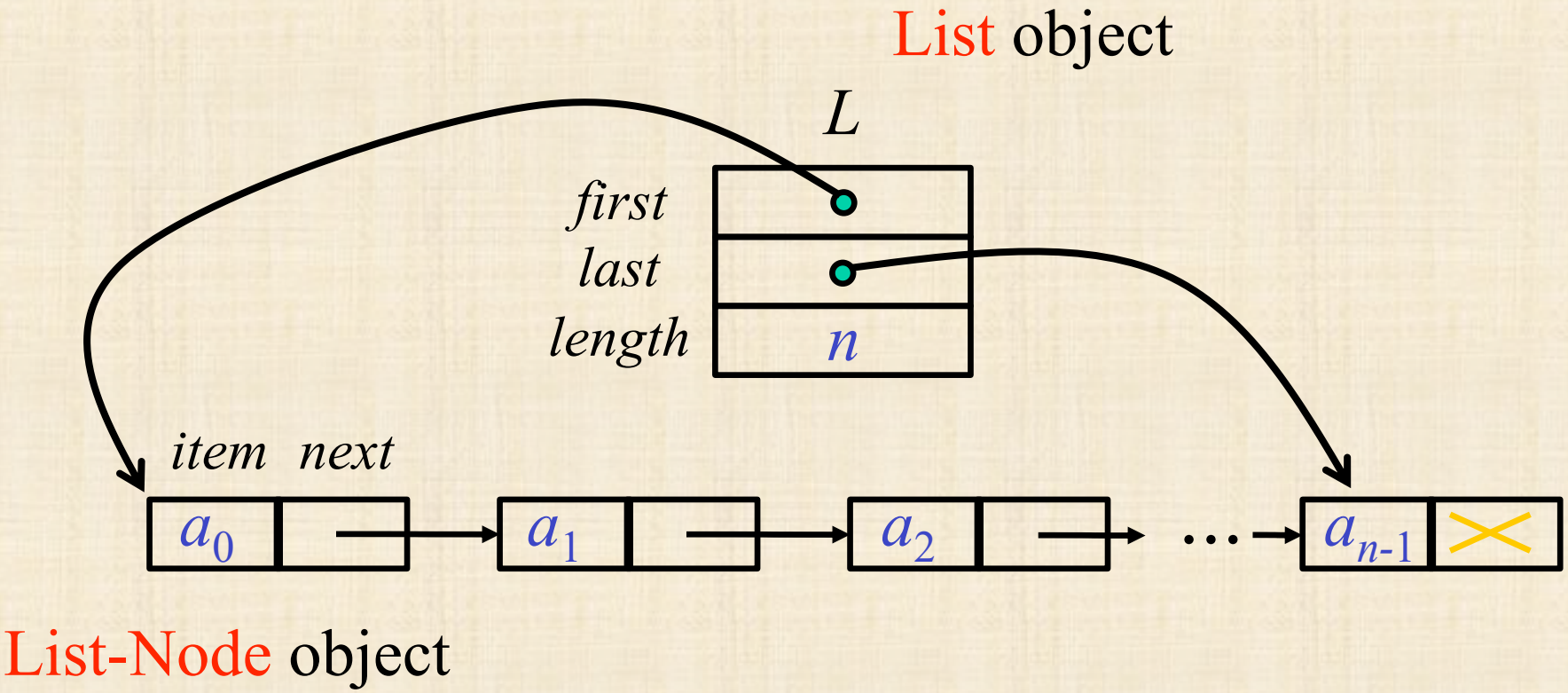
Summary - Circular array

List operation	Circular array
Insert/Delete-First Insert-Last	$O(1)$
Delete-Last	$O(1)$
Insert/Delete(i)	$O(\min\{i+1, n-i+1\})$
Retrieve(i)	$O(1)$
Concatenation	$O(\min\{n_1, n_2\}+1)$

Implementing Sequence using Singly Linked List



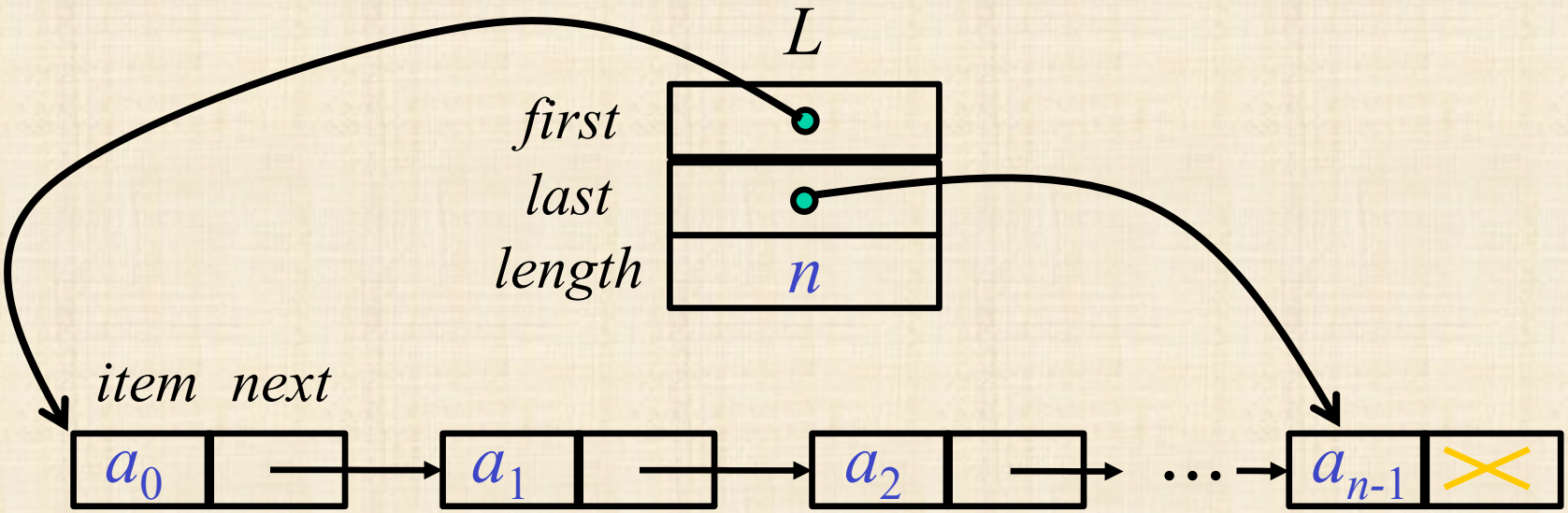
Implementing Sequence using Singly Linked List



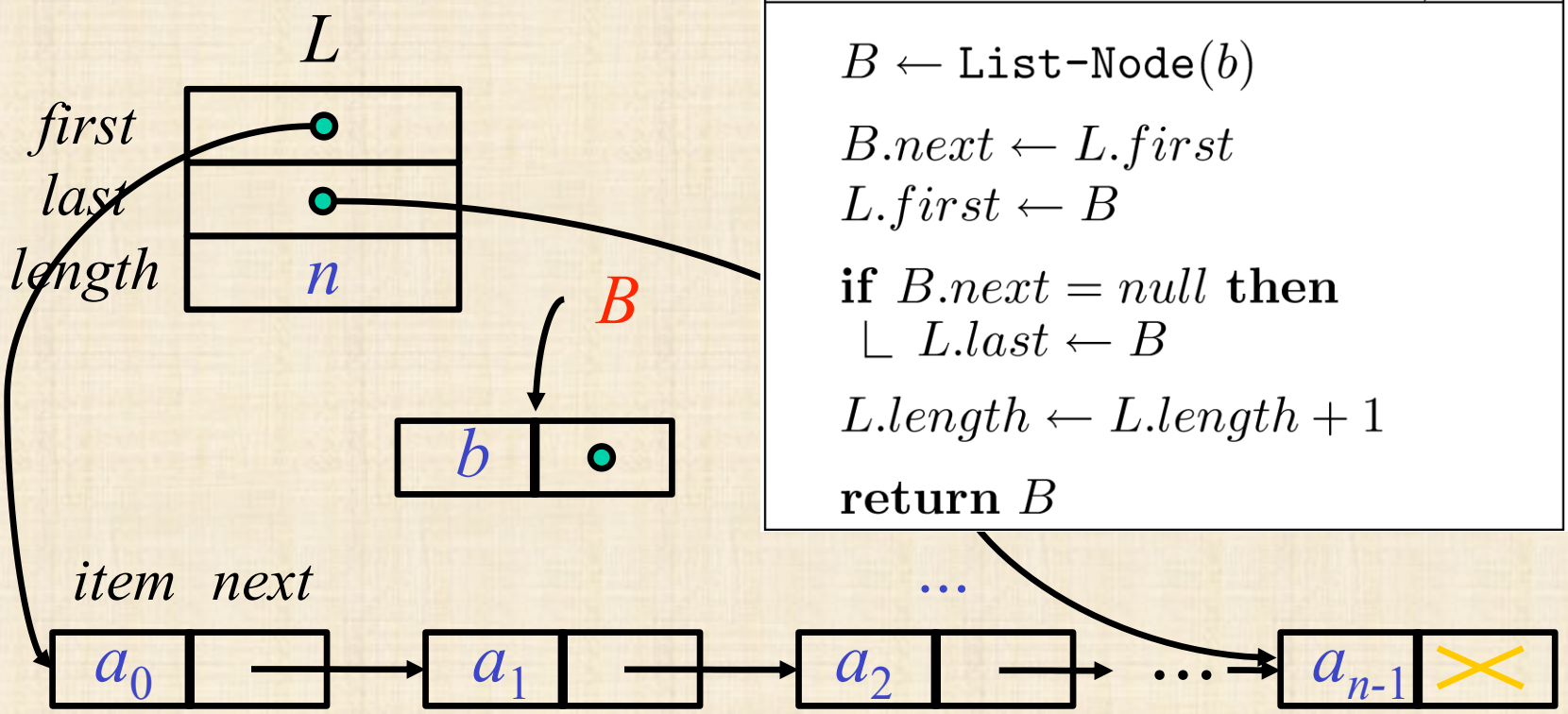
Implementing Sequence using Singly Linked List

No efficient O(1) random access to elements (for instance, cannot binary search) BUT:

- unbounded size
- no need to allocate space in advance
- scan in O(n) time
- concatenate lists in O(1) time



Insert-First with Singly Linked List



Function `Insert-First`(L, b)

$B \leftarrow$ `List-Node`(b)

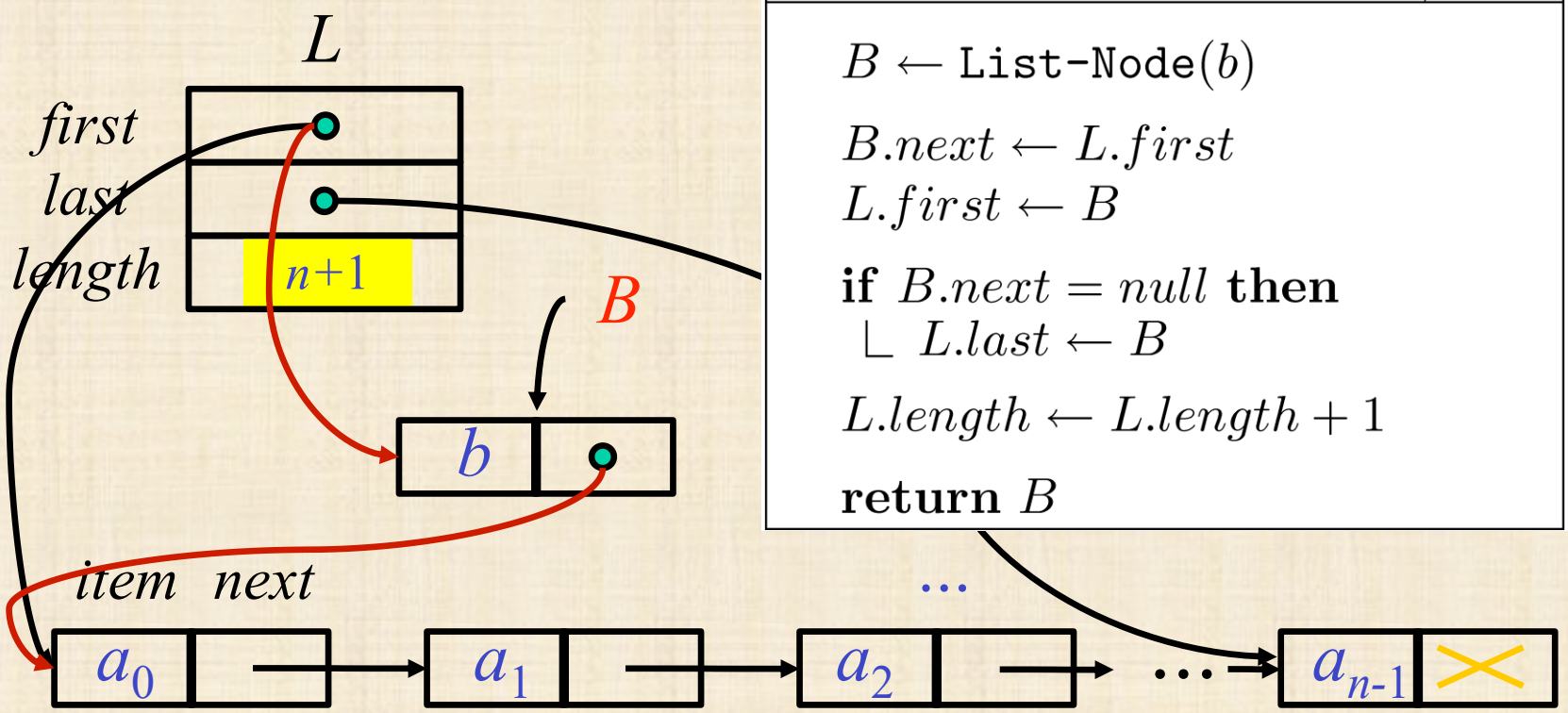
$B.next \leftarrow L.first$
$L.first \leftarrow B$

if $B.next = null$ **then**
  $L.last \leftarrow B$

$L.length \leftarrow L.length + 1$

return B

Insert-First with Singly Linked List



Function `Insert-First`(L, b)

$B \leftarrow$ `List-Node`(b)

$B.next \leftarrow L.first$
$L.first \leftarrow B$

if $B.next = null$ **then**
  $L.last \leftarrow B$

$L.length \leftarrow L.length + 1$

return B

Insert-Last and Delete-First– very similar, also O(1) time

Unfortunately, Delete and Delete-Last require O(n) time

Retrieve with Singly Linked List

Function `Retrieve`(L, i)

return `Retrieve-Node`$(L, i).item$

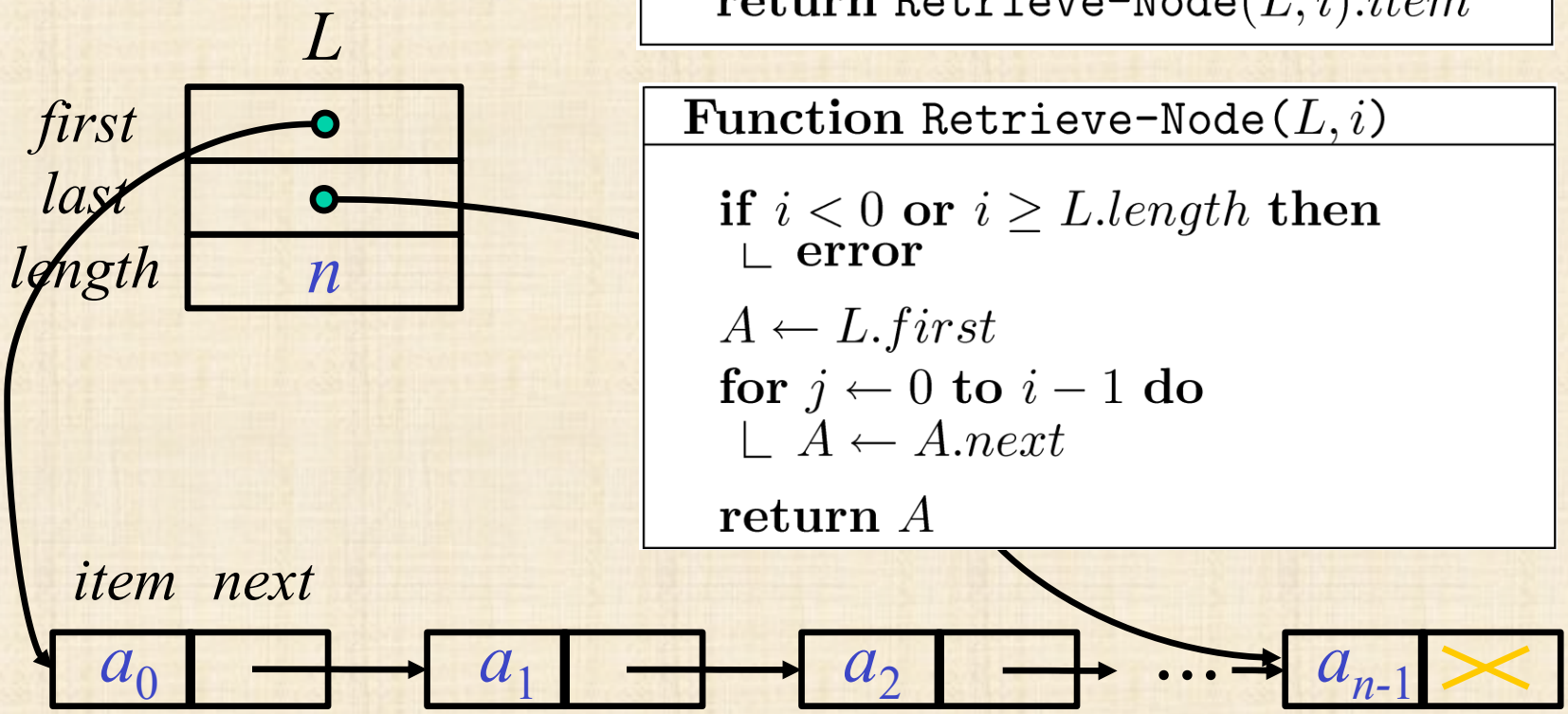
Function `Retrieve-Node`(L, i)

if $i < 0$ **or** $i \geq L.length$ **then**
  ∟ **error**

$A \leftarrow L.first$

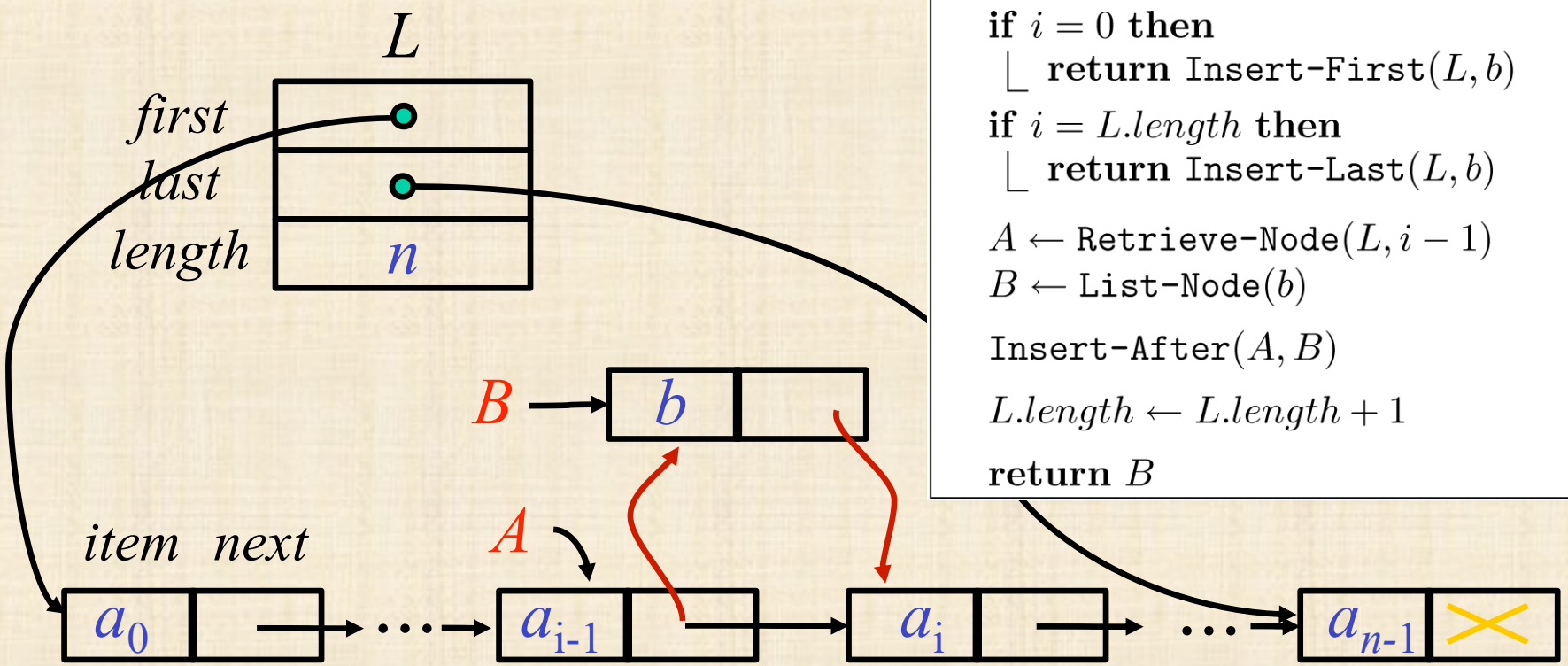
for $j \leftarrow 0$ **to** $i - 1$ **do**
  ∟ $A \leftarrow A.next$

return A



Retrieving the i-th item takes $O(i+1)$ time

Insert with Singly Linked List



Function `Insert`(L, i, b)

if $i < 0$ **or** $i > L.length$ **then**
  error

if $i = 0$ **then**
  return `Insert-First`(L, b)

if $i = L.length$ **then**
  return `Insert-Last`(L, b)

$A \leftarrow$ `Retrieve-Node`$(L, i-1)$
$B \leftarrow$ `List-Node`(b)

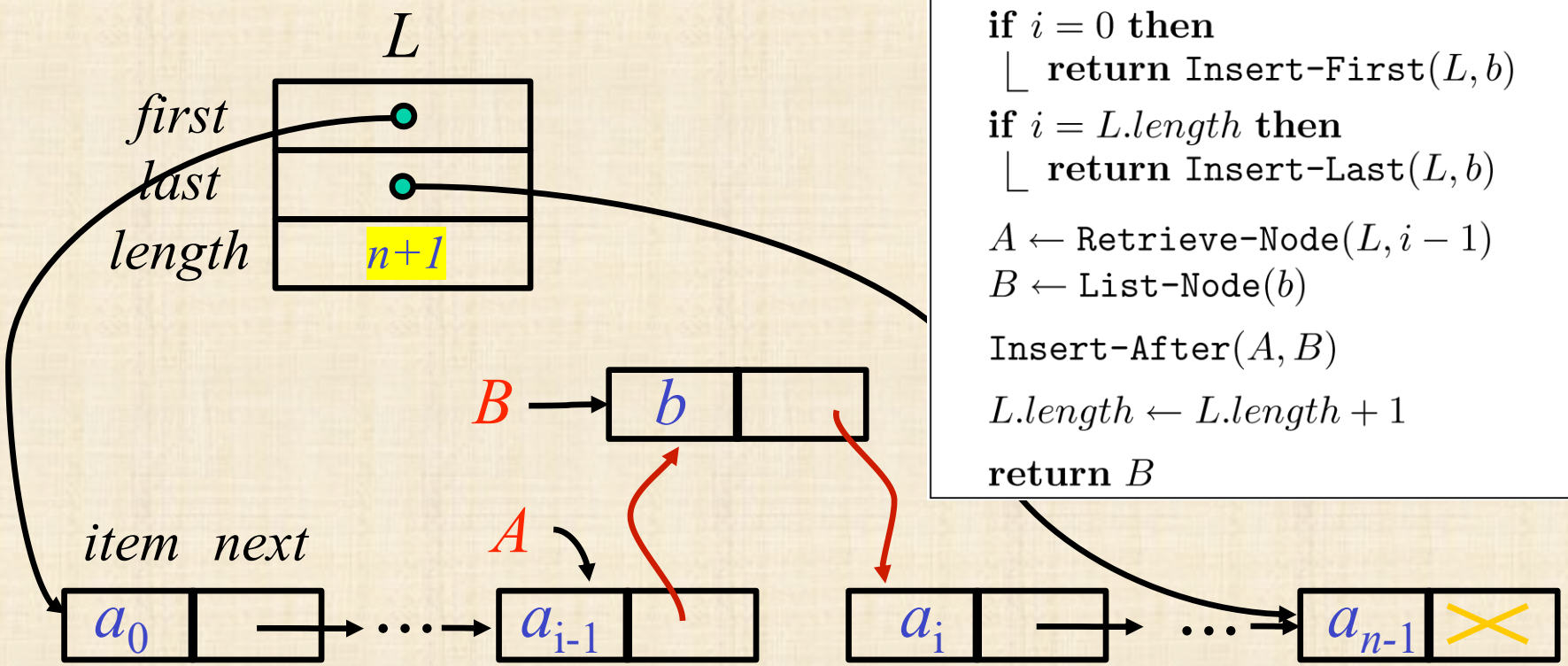
`Insert-After`(A, B)

$L.length \leftarrow L.length + 1$

return B

Inserting an item into position i takes $O(i+1)$ time

Insert with Singly Linked List



Function `Insert`(L, i, b)

if $i < 0$ **or** $i > L.length$ **then**
  error

if $i = 0$ **then**
  return `Insert-First`(L, b)

if $i = L.length$ **then**
  return `Insert-Last`(L, b)

$A \leftarrow$ `Retrieve-Node`$(L, i-1)$
$B \leftarrow$ `List-Node`(b)

`Insert-After`(A, B)

$L.length \leftarrow L.length + 1$

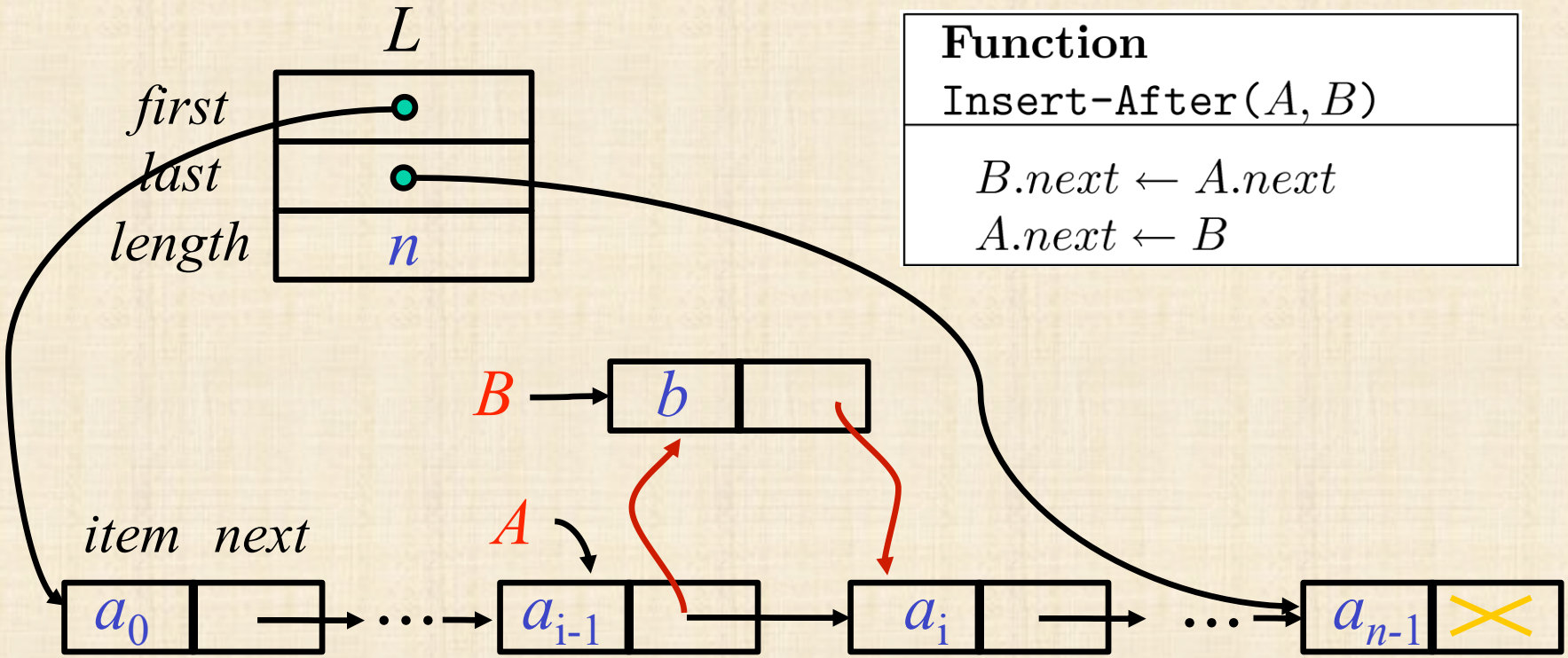
return B

Inserting an item into position i takes $O(i+1)$ time

Inserting a node

(After a given node)

Insert-After(A,B) – Insert B after A



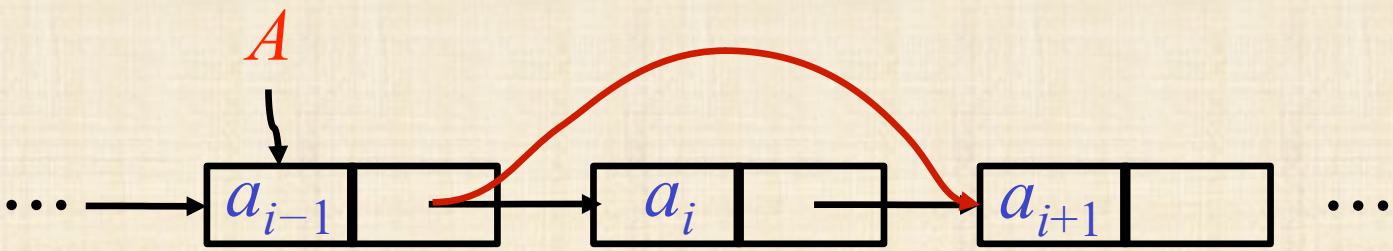
Function
`Insert-After`(A, B)
$B.next \leftarrow A.next$ $A.next \leftarrow B$

Inserting an item into position i takes O(i+1) time

Deleting a node

(After a given node, not the last one)

Delete-After(A) – Delete the node following A



Function `Delete-After`(A, B)
$A.next \leftarrow A.next.next$

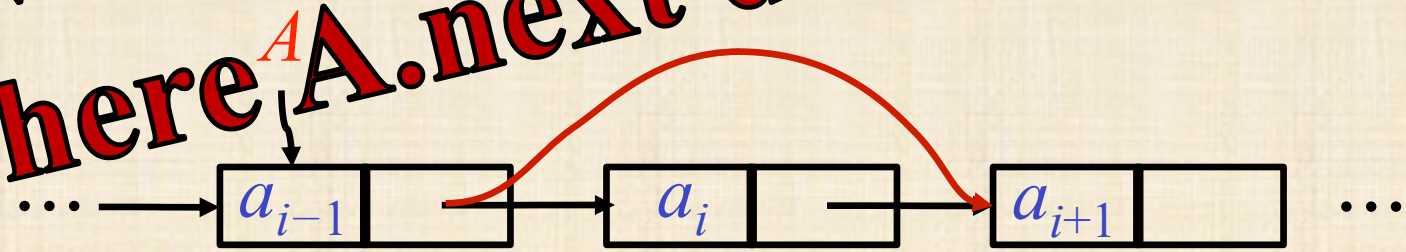
What happens to the node removed?

Deleting a node

(After a given node, not the last one)

We did not cover the case
Where A.next does not exists

Delete-After(A) – Delete the node following A



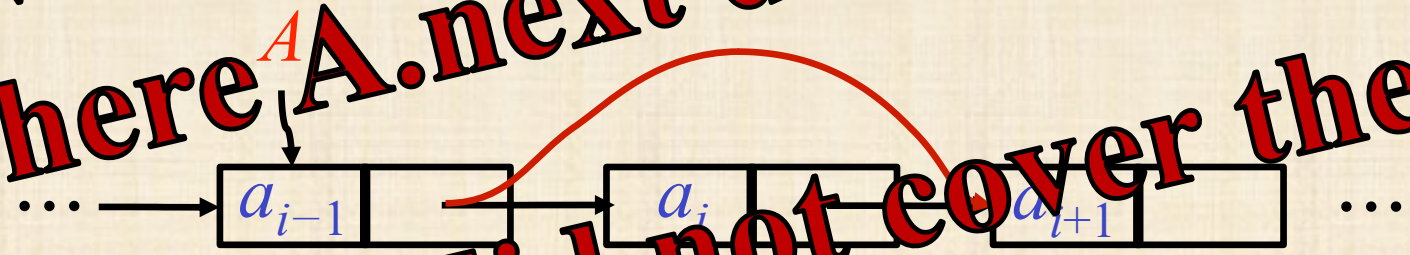
Function `Delete-After`(A, B)
$A.next \leftarrow A.next.next$

What happens to the node removed?

Deleting a node

(After a given node, not the last one)

Delete_After(A) – Delete the node following A



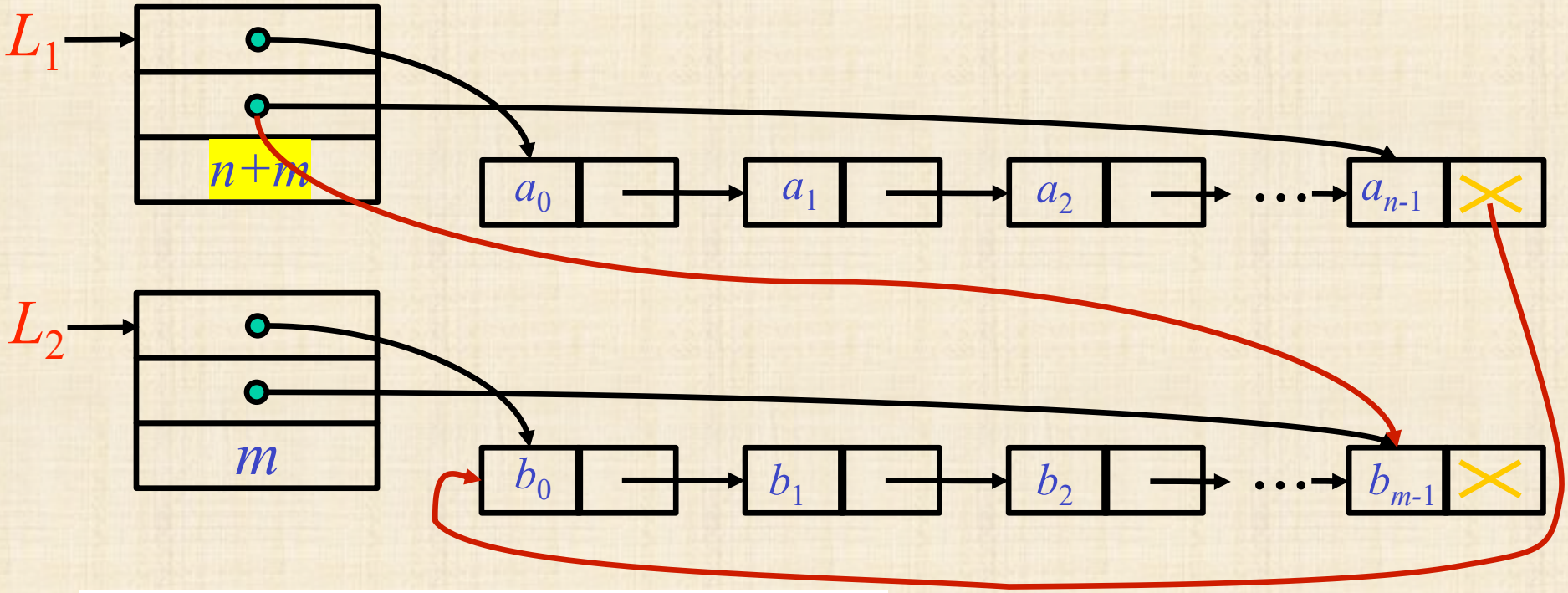
Function Delete_After(A, B)
$A.next \leftarrow A.next.next$

What happens to the node removed?

We did not cover the case
Where A.next does not exists
We Also did not cover the
Envelope code
(eg, L.length, edge cases)

Concatenating Lists

Concat(L_1,L_2) – Return a list consisting of the elements of L_1 followed by the elements of L_2.



Function `Concat`(L_1, L_2)

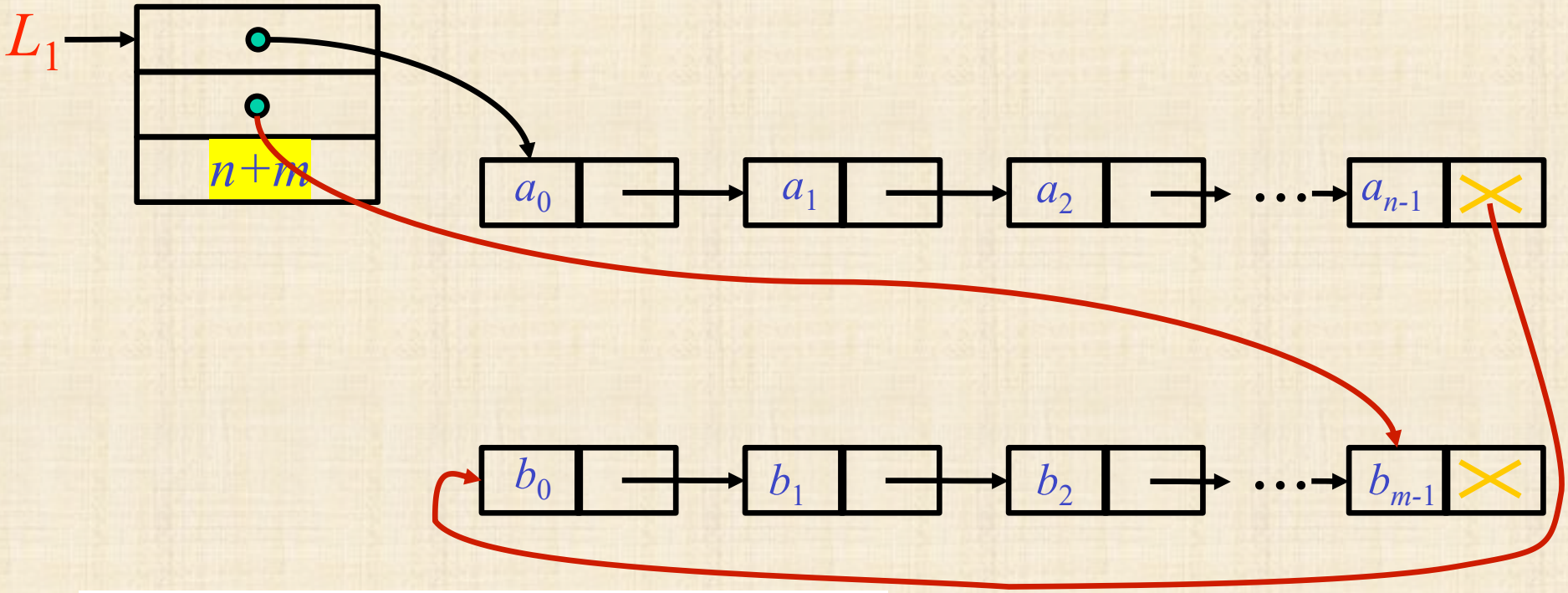
$L_1.last.next \leftarrow L_2.first$
$L_1.last \leftarrow L_2.last$
$L_1.length \leftarrow L_1.length + L_2.length$

O(1) time!

(What happened to L_2?)

Concatenating Lists

Concat(L_1,L_2) – Return a list consisting of the elements of L_1 followed by the elements of L_2.



Function `Concat`(L_1, L_2)

$L_1.last.next \leftarrow L_2.first$
$L_1.last \leftarrow L_2.last$
$L_1.length \leftarrow L_1.length + L_2.length$

O(1) time!

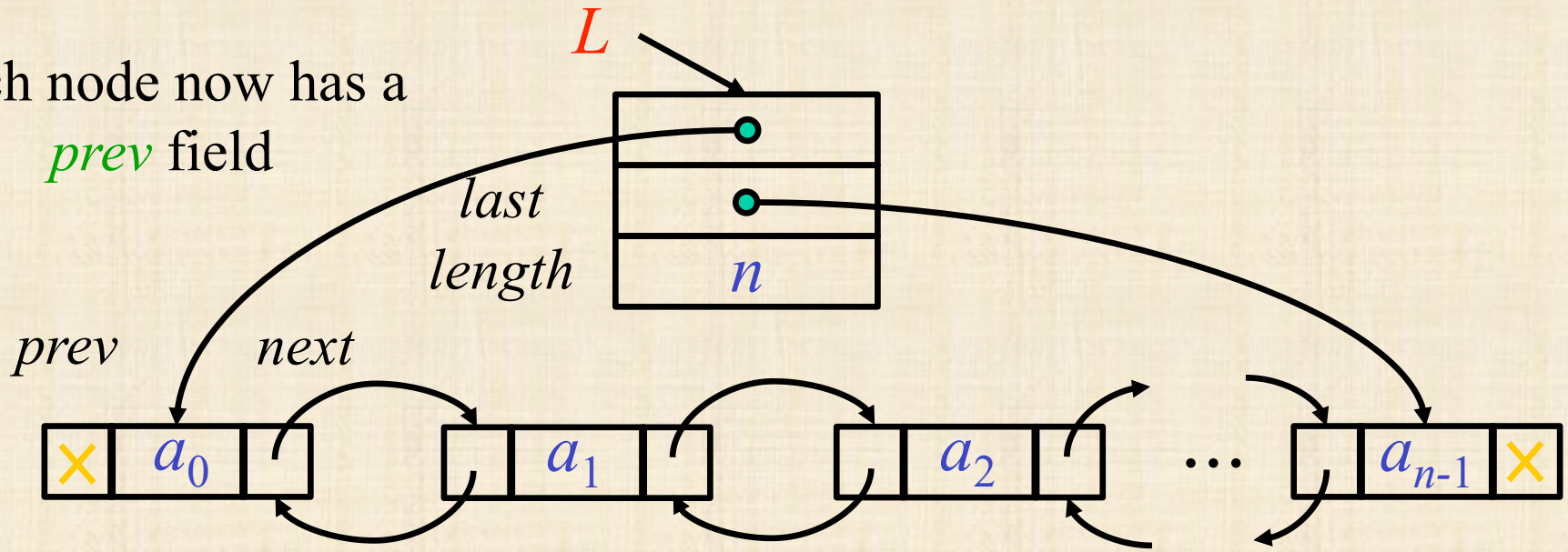
(What happened to L_2?)

Implementing Sequence using Doubly Linked List

Insert/Delete-Last in O(1) time

Concatenation also in O(1) time

Each node now has a *prev* field

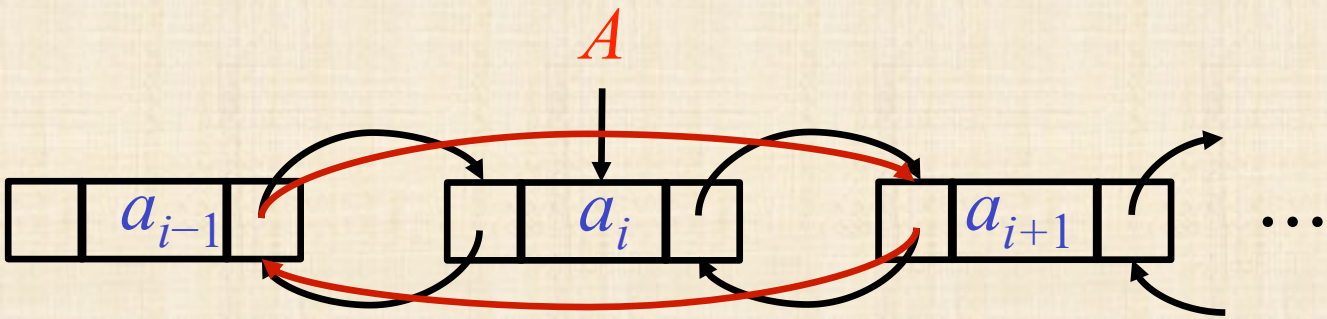


Careful, first and last elements are special.

Deleting a node from a Doubly Linked List

Each node now has a *prev* field

Delete(A) – Delete node A from its list



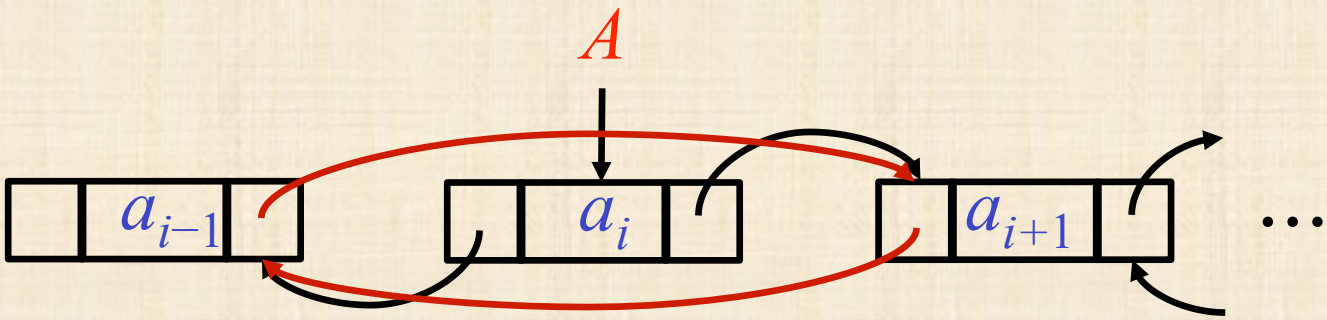
Function `Delete`(A)
$A.prev.next \leftarrow A.next$ $A.next.prev \leftarrow A.prev$

Note: A itself not changed!

Deleting a node from a Doubly Linked List

Each node now has a *prev* field

Delete(A) – Delete node A from its list



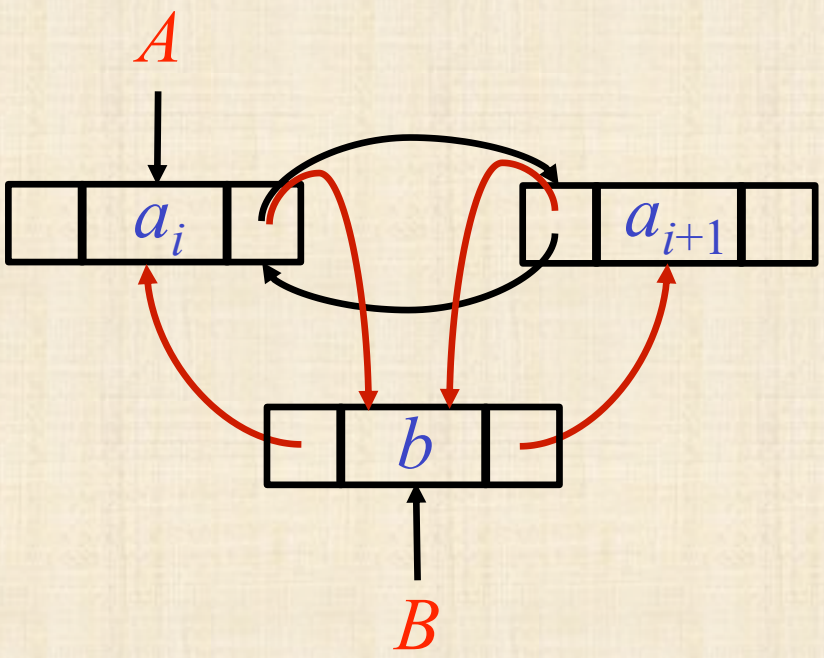
Function `Delete`(A)
$A.prev.next \leftarrow A.next$ $A.next.prev \leftarrow A.prev$

Note: A itself not changed!

We also did not take care of L.Length

Inserting a node into a Doubly Linked List

Insert-After(A,B) – Insert node B after node A



Function
`Insert-After`(A, B)

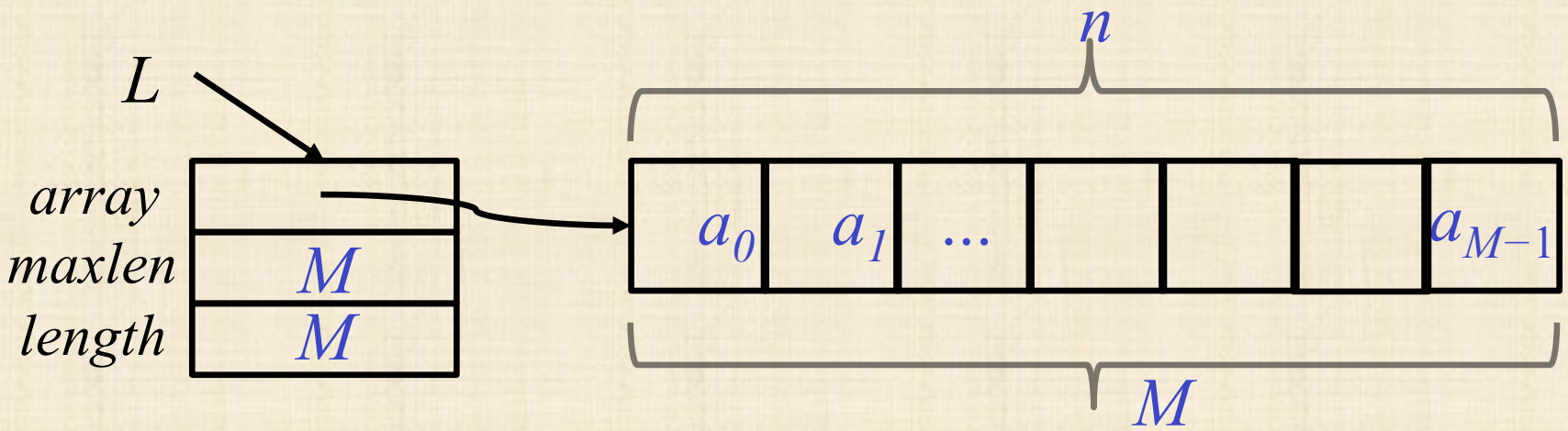
$B.prev \leftarrow A$
$B.next \leftarrow A.next$

$A.next \leftarrow B$
$B.next.prev \leftarrow B$

Circular arrays vs. Linked Lists

	Circular arrays	Doubly Linked lists
Insert/Delete-First	$O(1)$	$O(1)$
Insert/Delete-Last	$O(1)$	$O(1)$
Insert/Delete(i)	$O(\min\{i+1, n-i+1\})$	$O(\min\{i+1, n-i+1\})$
Retrieve(i)	$O(1)$	$O(\min\{i+1, n-i+1\})$
Concatenation	$O(\min\{n_1, n_2\}+1)$	$O(1)$

Implementing lists using (circular) arrays with resizing



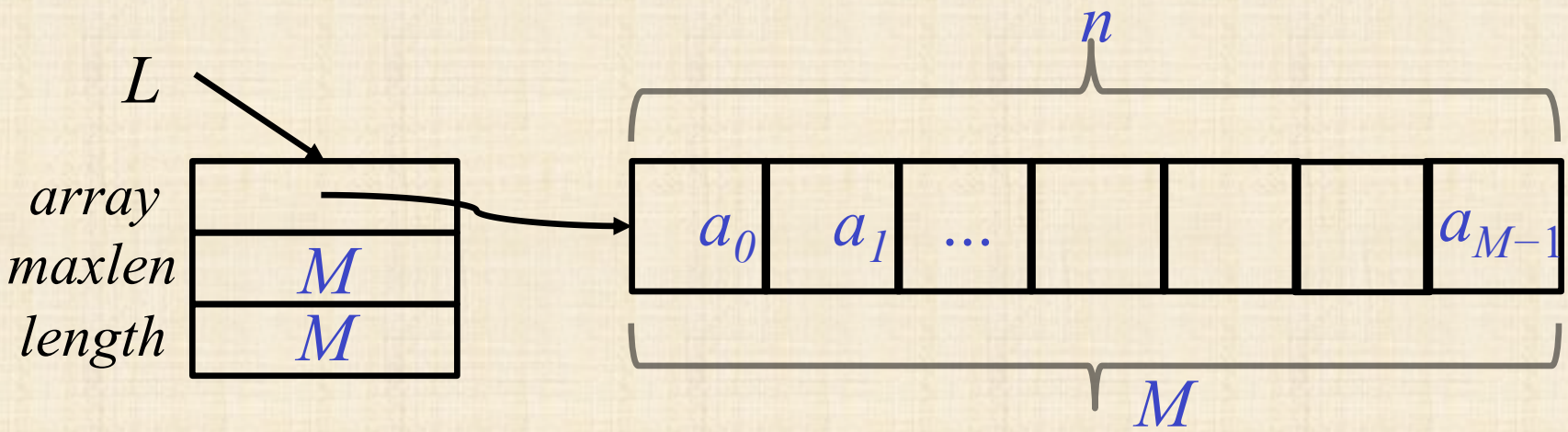
What do we do when the array is full?

We cannot extend the array.

Allocate a larger array and copy

What should be the size of the new array?

Implementing lists using (circular) arrays with resizing



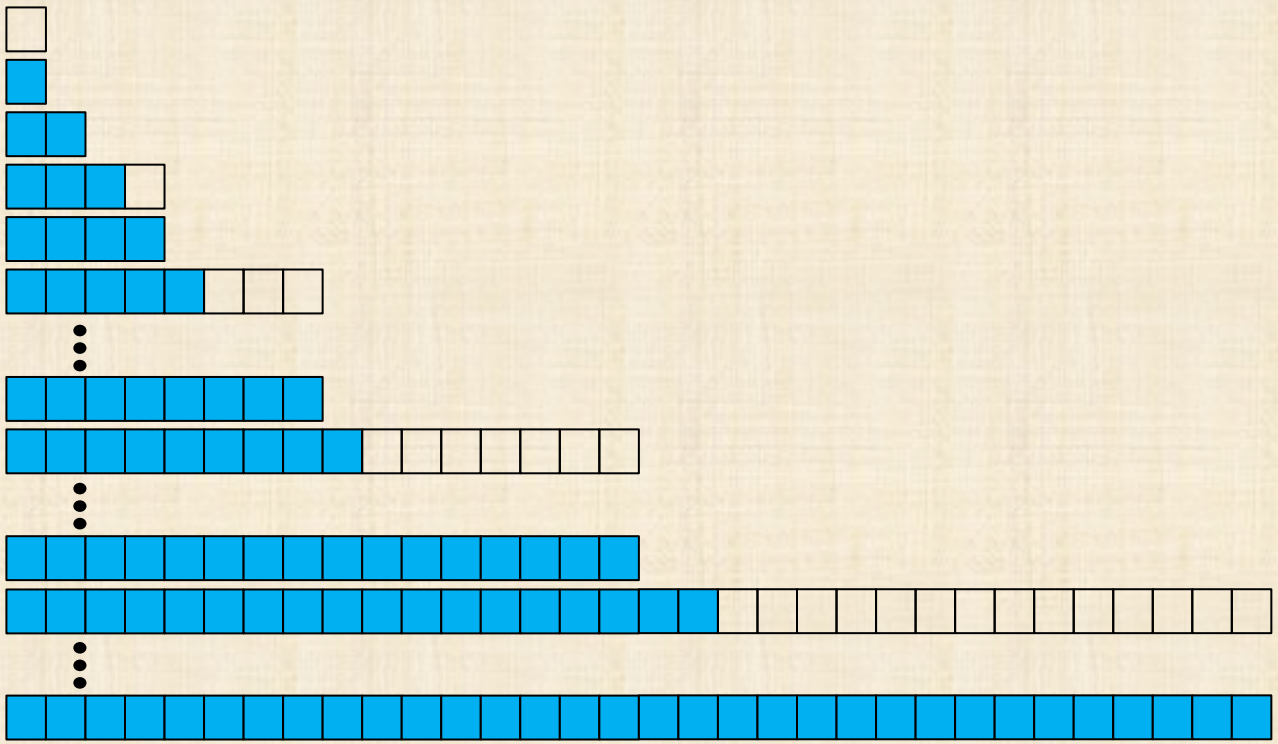
If we start with an empty array and increase its length by 1, then the time to do n Insert-Last is about:

$$1 + 2 + \ldots + n = \frac{n(n+1)}{2} = O(n^2)$$

Implementing lists using (circular) arrays with doubling

When the array is full, double its size and copy.

What is the cost of n Insert-Last operations?

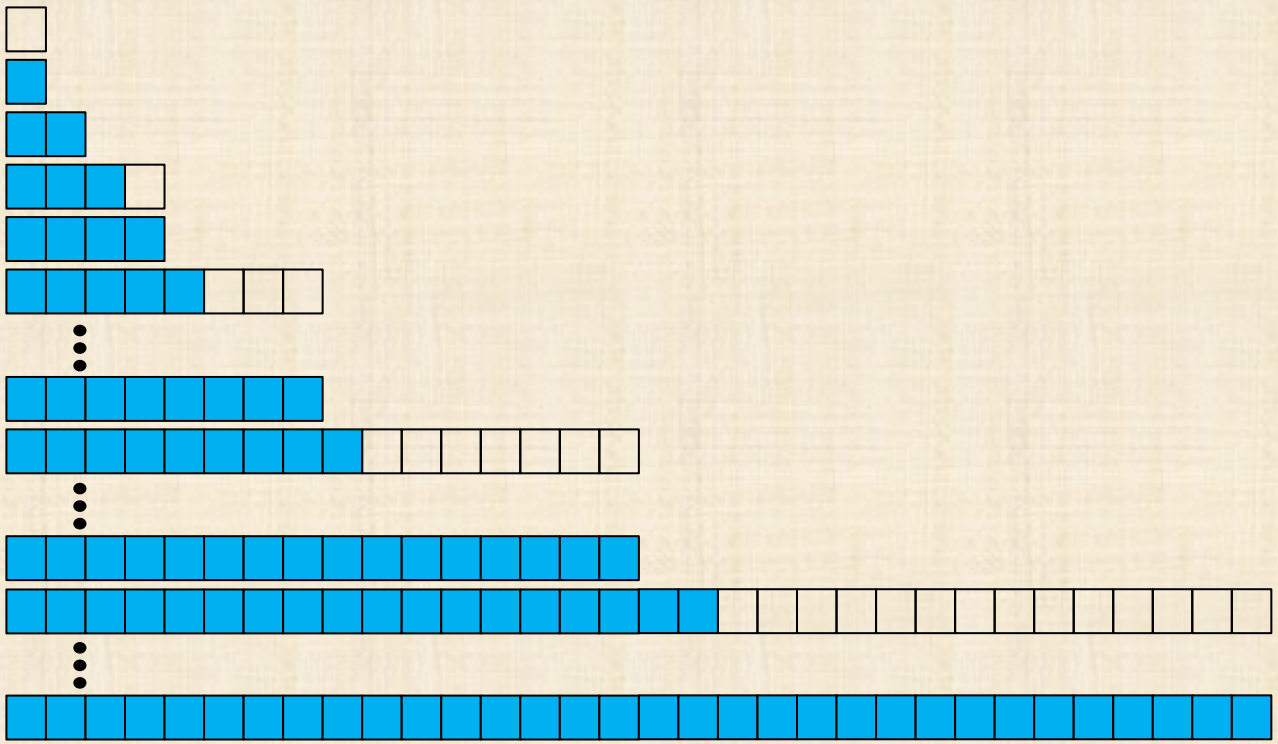


inserts	copies
0	0
1	0
2	1
3	2
4	0
5	4
⋮	
8	0
9	8
⋮	
16	0
17	16
⋮	
32	0

Implementing lists using (circular) arrays with doubling

When the array is full, double its size and copy.

What is the cost of n Insert-Last operations?



inserts	copies
0	0
1	0
1	1
1	2
1	0
1	4
⋮	
1	0
1	8
⋮	
1	0
1	16
⋮	
1	0
n	1+2+4+..+n/2

Implementing lists using (circular) arrays with doubling

When the array is full, double its size and copy.

What is the cost of n Insert-Last operations?

Assume, for simplicity that $n=2^k$

$$\underbrace{(1+1+1+\ldots+1)}_{\text{cost of n insertion}} + \underbrace{(1+2+4+\ldots+\frac{n}{2})}_{\text{cost of copying}} = n+(n-1) = O(n)$$

We say that The amortized cost of each operation is O(1)

Worst-case bounds

worst(op) = maximum time for the operation *op*.

The time to serve a sequence of n operations:

$$Time(n \times op) \leq n \times worst(op)$$

Sometimes, this bound is very loose:

$$worst(\texttt{Insert-Last}) = O(n)$$

$$Time(n \times \texttt{Insert-Last}) = O(n)$$

Amortized analysis

We say that *amort*(*op*) is an amortized bound on the cost of operation *op* iff for every *n* and every sequence of *n* operations *op*

$$amort(op) \geq Time(n \times op) \ / \ n$$

$$worst(\texttt{Insert-Last}) = O(n)$$

$$amort(\texttt{Insert-Last}) = O(1)$$

$$Time(n \times \texttt{Insert-Last}) \leq n \cdot amort(\texttt{Insert-Last}) = O(n)$$

Amortized analysis

amortized analysis bounds the average running time per operation over the worst-case sequence of operations.

Amortized analysis is useful when what we really care about is the time of the entire sequence, not the time of each individual operation.

In other cases, such as real-time applications, where we need each individual operation to be fast, we should use worst case analysis.

Implementing lists using (circular) arrays with doubling

When the array is full, double its size and copy.

What is the cost of n Insert-Last operations?

Assume, for simplicity that $n=2^k$

$$\underbrace{(1+1+1+\ldots+1)}_{\text{cost of n insertion}} + \underbrace{(1+2+4+\ldots+\frac{n}{2})}_{\text{cost of copying}} = n+(n-1) = O(n)$$

The amortized cost of each operation is $O(1)$

The worst-case cost is $O(n)$

Implementing lists using (circular) arrays with doubling

When the array is full, double its size and copy.

$n=2^k$ seems to be a 'fortunate' case.

In general, $n=2^k+r$ for $0 \leq r < 2^k$

$$\underbrace{(1+1+1+\ldots+1)}_{\text{cost of n insertion}} + \underbrace{(1+2+4+\ldots+2^k)}_{\text{cost of copying}} = n + (2^{k+1}-1) = O(n)$$

$$\underbrace{\underbrace{n + (2^{k+1}-1)}_{\leq n + 2n - 1}}_{3n-1}$$

The amortized cost of each operation is still $O(1)$

Amortized Analysis
≠
Average case

- **amortized analysis** bounds the average running time per operation over the worst-case sequence of operations.
- **Average case analysis** bounds the average time of a single operation over all possible inputs

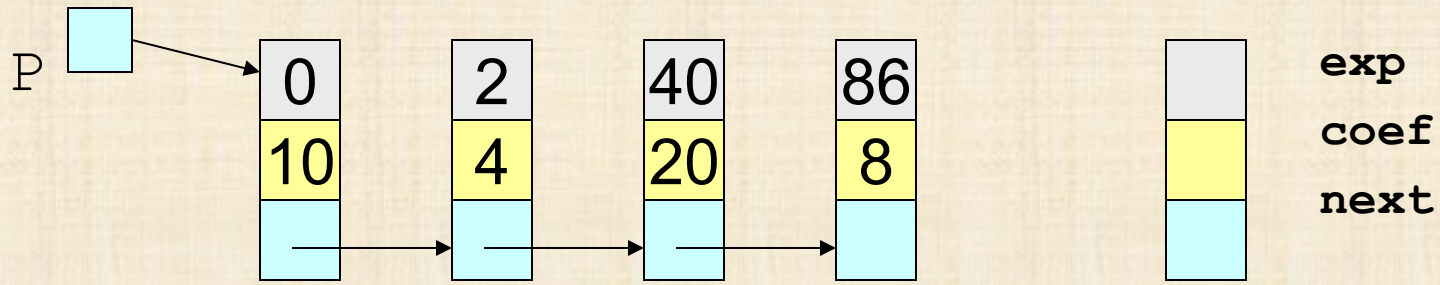
Application Example

- Add sparse polynomial
  - $Q(x)=7x + 10x^2 - 8x^{86}$
  - $P(x)=10 + 4x^2 + 20x^{40} + 8x^{86}$
  - $R(x)=Q(x)+P(x)= \Sigma(q_i+p_i)x^i$
- Possible data structure
  - Arrays

  - Linked lists

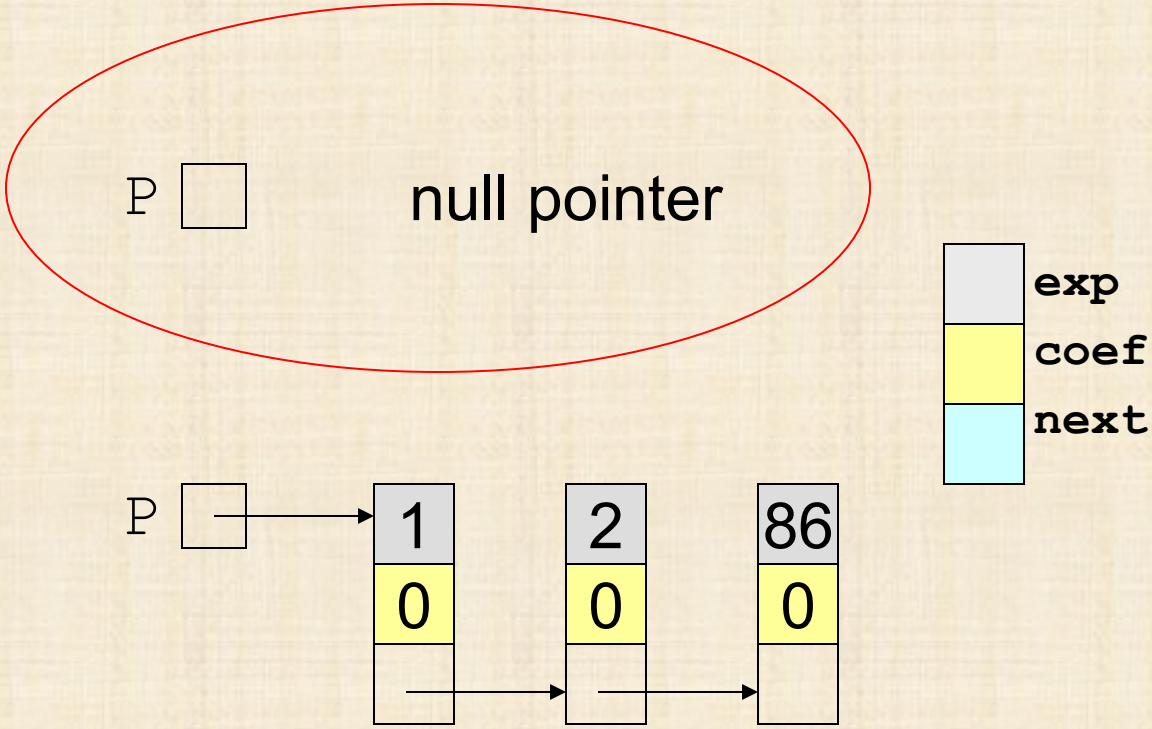
Application example: Sparse Polynomials

- $10 + 4\,x^2 + 20\,x^{40} + 8\,x^{86}$



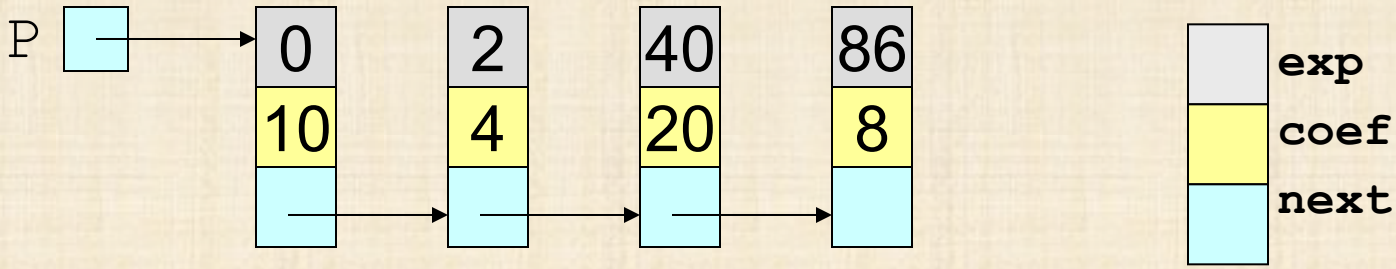

```
poly : {
   exp : integer,
   coef : integer,
   next : poly pointer
}
```

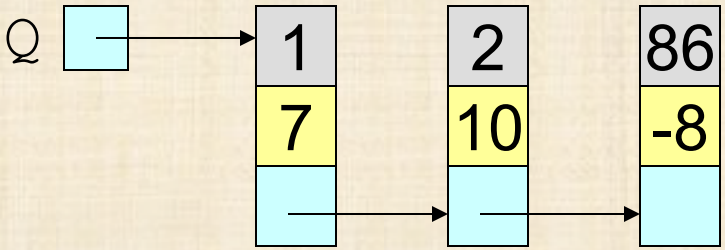
Identically Zero Polynomial



Addition of Polynomials

$P(x)= 10 + 4\,x^2 + 20\,x^{40} + 8\,x^{86}$

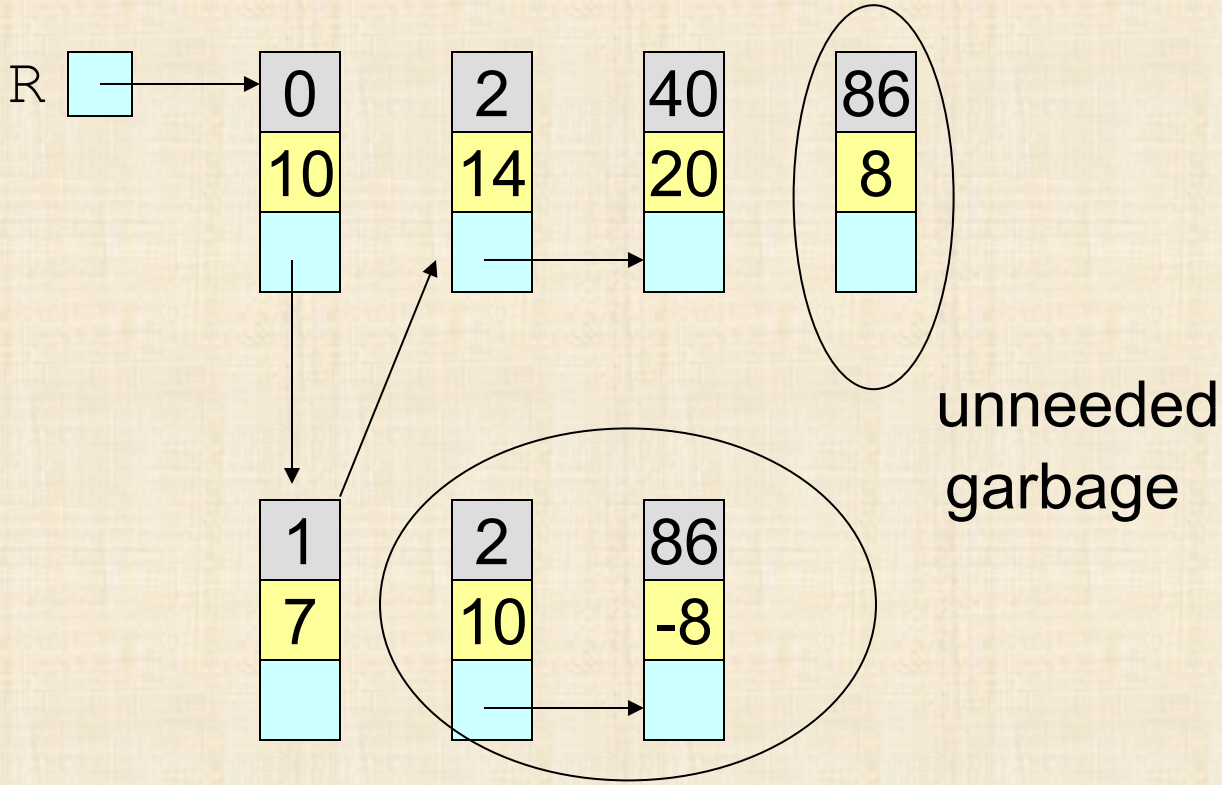




$Q(x)=7\,x + 10\,x^2 - 8\,x^{86}$

Result

$P(x)= 10 + 4\ x^2 + 20\ x^{40} + 8\ x^{86}$



$Q(x)=7\ x + 10\ x^2 - 8\ x^{86}$

Summary

- ADT: An abstract mathematical specification of objects, with operations defined on them
- Data Structures:
  - Representation of objects of an Abstract Data Type (ADT)
  - Algorithms manipulate the data structures to implement the operations of the ADT
- Examples

Course Overview

- Introduction to basic data structures
  - Understand the data structures
  - How to implement them
  - Analyze the algorithms that use them
  - Know when to apply them
- Practice design and analysis of data structures
- Practice with these data structures by writing your own code

Next Lecture

- Trees
- Heaps

Supplementary Material

- The next set of slides contains the pseudocode of adding sparse polynomials
- Note: we do not cover these slides in class. It is an additional example you may find useful / interesting

Recursive Addition

```
Add (P, Q)        % pointers to nodes
case {
        P == null      :      R ← Q
        Q == null      :      R ← P
    P.exp < Q.exp      :      R ← P
                              R.next ← Add(P.next,Q);
    P.exp > Q.exp      :      R ← Q
                              R.next ← Add(P,Q.next);
    P.exp == Q.exp     :      R ← P
                              R.coef ← P.coef + Q.coef
    }                         R.next ← Add(P.next,Q.next)
    return R
```

What is the problem?

cont.: Solution for Polyn. Addition

% The terms with coef = 0 are removed from the result

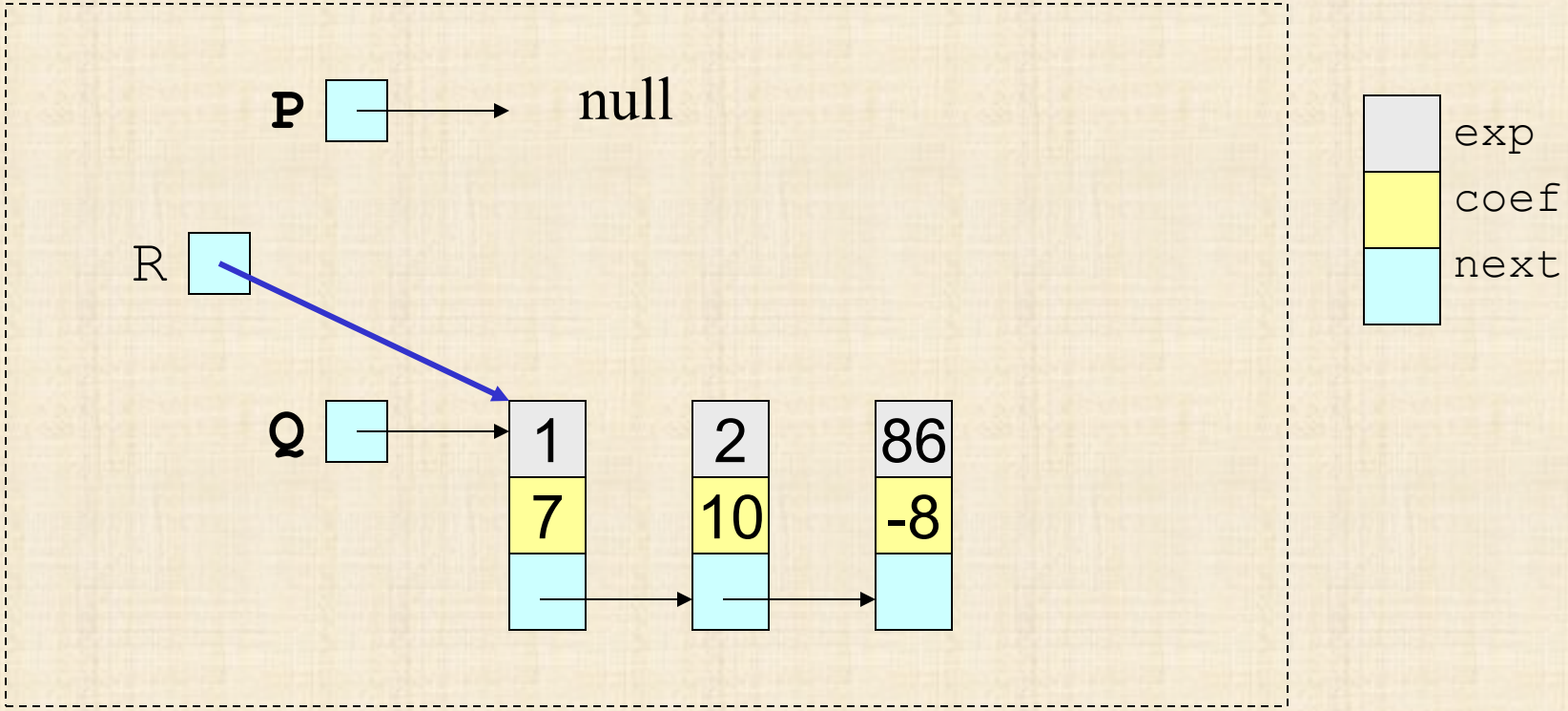
```
P.exp == Q.exp :     R ← P ;

                     R.coef ← P.coef + Q.coef ;
                     if R.coef = 0 then
                          R ← Add(P.next,Q.next);
                     else
                          R.next ← Add(P.next,Q.next);
```

Notes on Addition

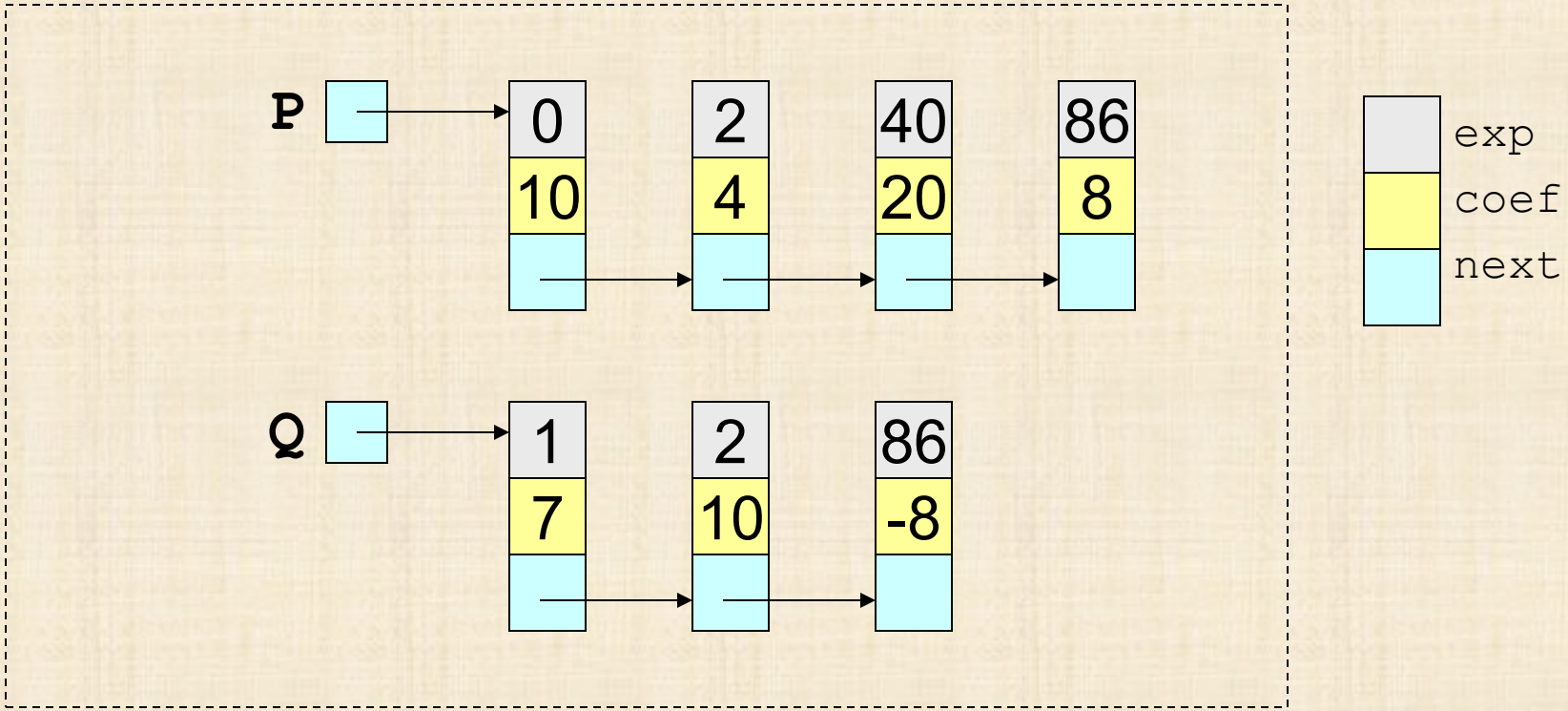
- Addition is destructive, that is, the original polynomials are gone after the operation
- We didn’t salvage “garbage” nodes

Example 1: R=Add(P,Q)

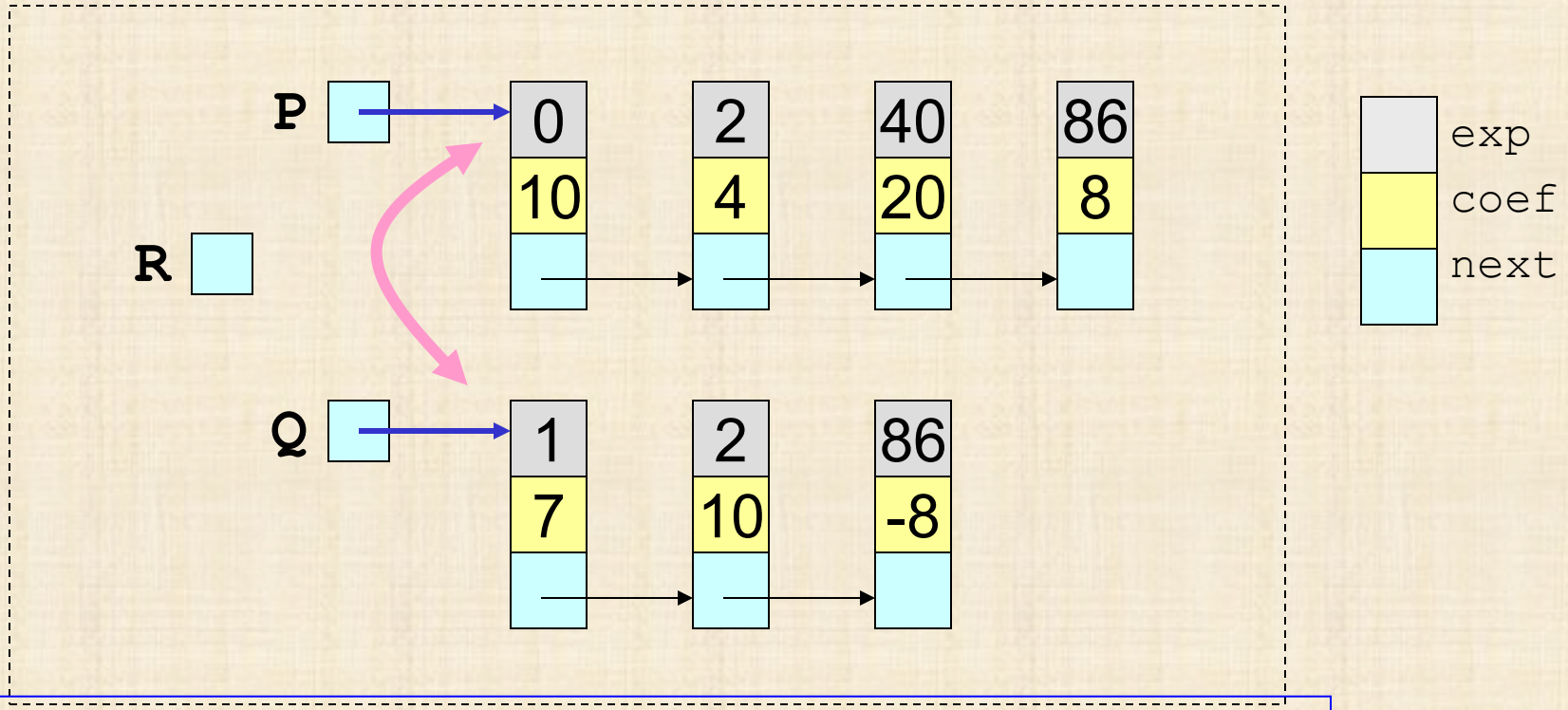



```
case {
      P == null:          R ← Q
      Q == null: R ← P
   …
```

Example 2: R=Add(P,Q)

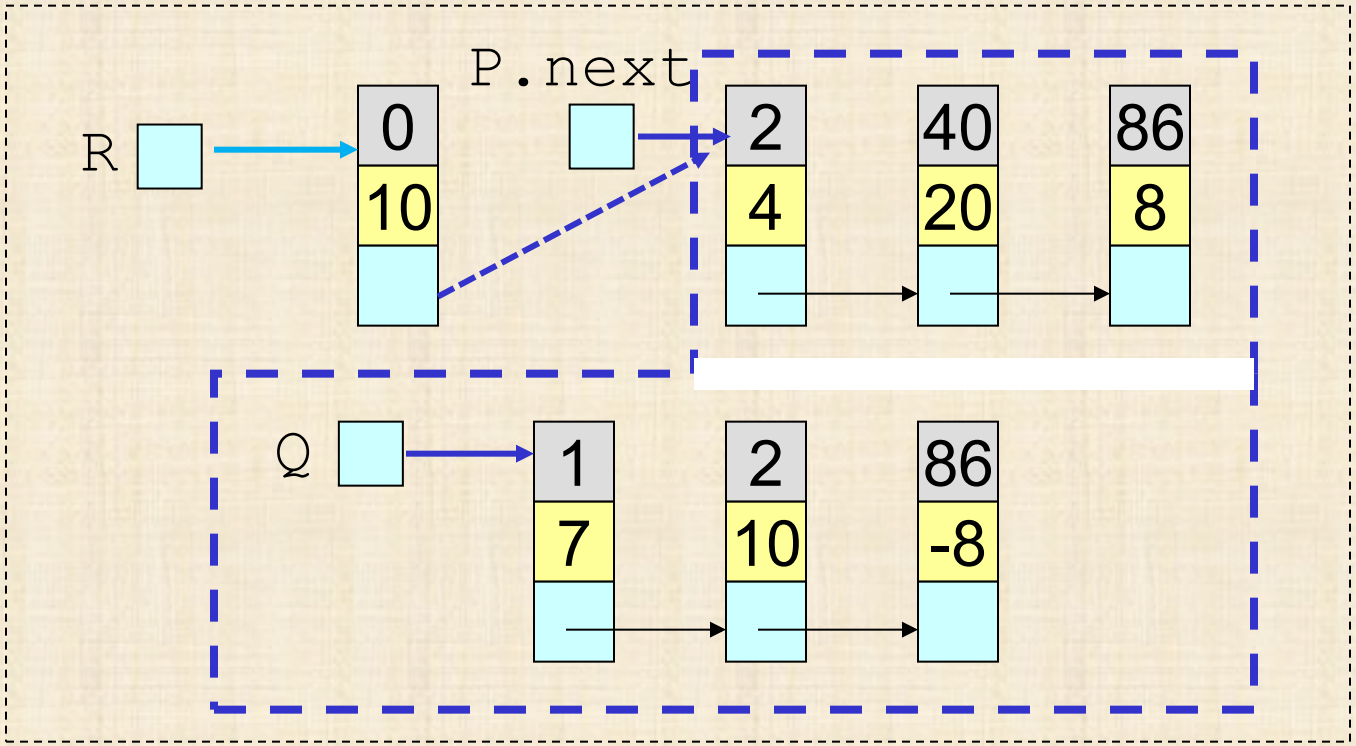


Example 2 (first call)




```
case {…
        P.exp < Q.exp : R ← P
                        R.next ← Add(P.next,Q);
…}
```

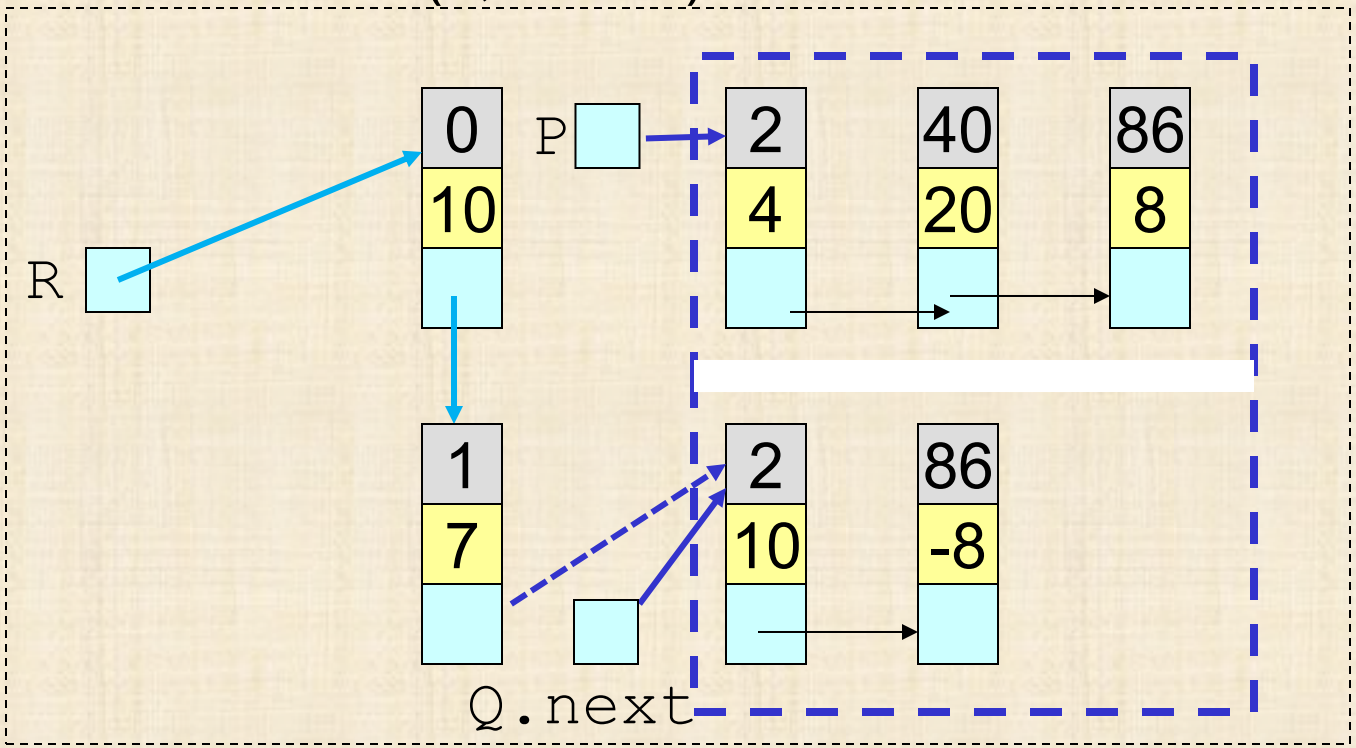
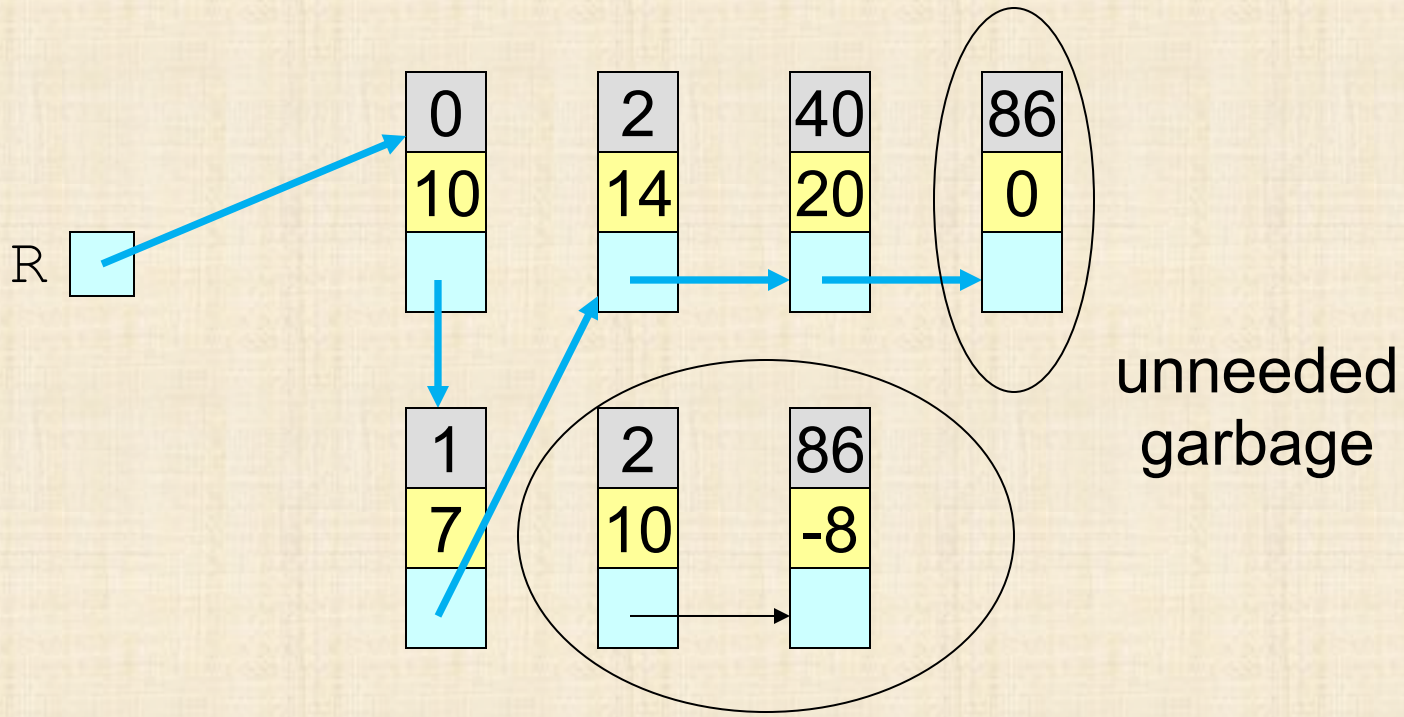
The Recursive Call




```
case {…
        Q.exp < P.exp : R ← Q
                        R.next ← Add(P,Q.next);
…}
```

The Recursive Call

R.next=Add(P,Q.next)



The final picture